

Topology Optimization of Heat Transfer Applications Operating in the Turbulent Regime

Tiebert Lefebure

Thesis submitted for the degree of
Master of Science in Mechanical
Engineering

Supervisors

Prof. Niels Horsten
Prof. Maarten Blommaert

Assessors

Prof. Elke Deckers
Olivier Renders

Assistant-supervisor

Esteban Foronda Obando

© 2026 KU Leuven – Faculty of Engineering Science

Published by Tiebert Lefebure,

Faculty of Engineering Science, Kasteelpark Arenberg 1 bus 2200, B-3001 Leuven

All rights reserved. No part of the publication may be reproduced in any form by print, photoprint, microfilm, electronic or any other means without written permission from the publisher. This publication contains the study work of a student in the context of the academic training and assessment. After this assessment no correction of the study work took place.

Preface

This thesis marks the culmination of my Master of Science in Mechanical Engineering at KU Leuven. The journey of exploring computational fluid dynamics and mathematical optimization has been both intensely challenging and deeply rewarding. Bridging the gap between theoretical mathematics and practical engineering applications required countless hours of coding, debugging, and analyzing flow fields, but seeing the optimizer successfully carve out organic, aerodynamic fluid channels made the entire process worthwhile.

I would like to express my sincere gratitude to my supervisors, Prof. Niels Horsten and Prof. Maarten Blommaert, for providing me with the opportunity to work on this fascinating topic. Their academic guidance, critical insights, and continuous support were instrumental in shaping the direction and rigor of this research. A very special thanks goes to my assistant-supervisor, Esteban Foronda Obando, for his day-to-day patience, technical expertise, and willingness to help me navigate the complex numerical hurdles within FEniCS.

Furthermore, I would like to thank the assessors, Prof. Elke Deckers and Olivier Renders, for their time and valuable feedback in evaluating this work.

Finally, I want to thank my family and friends for their unwavering encouragement, patience, and perspective throughout the course of my studies.

Tiebert Lefebure

Contents

Preface	i
Abstract	iii
List of Figures and Tables	iv
List of Abbreviations and Symbols	vi
1 Introduction	1
1.1 Background and Motivation	1
1.2 Conceptual Overview of Density-Based Topology Optimization . . .	1
1.3 The Evolution of Fluid Topology Optimization	2
1.4 Objectives and Research Questions	4
1.5 Outline of the Thesis	5
2 Turbulence Modeling	6
2.1 Introduction to Fluid Dynamics and Turbulence	6
2.2 Reynolds-Averaged Navier-Stokes (RANS) Equations	7
2.3 Boundary Layer Theory and Near-Wall Treatment	8
2.4 The Spalart-Allmaras One-Equation Model	8
2.5 Evaluation of Curvature Effects and the Dean Number	9
2.6 Benchmark Studies in Ansys Fluent	11
2.7 Conclusion	13
3 Implementation and Verification of the Spalart-Allmaras Model in FEniCS	14
3.1 Introduction to FEniCS and the Finite Element Method	14
3.2 Numerical Solution Strategy for Fluid Dynamics	15
3.3 Detailed Formulation of the Spalart-Allmaras Model	17
3.4 Variational Formulation and FEniCS Implementation	19
3.5 General Solver Configuration and Numerical Parameters	20
3.6 Verification Benchmark 1: Channel Flow	20
3.7 Verification Benchmark 2: U-Bend Geometry	22
3.8 Conclusion	24
4 Topology Optimization for Turbulent Flows: Methodology	26
4.1 Introduction	26
4.2 The Fictitious Domain Method and Brinkman Penalization	27
4.3 Penalization of the Governing Equations	27

4.4	Sensitivity Analysis and the Frozen Turbulence Assumption	29
4.5	The Coupled "Semi-Frozen" Adjoint Formulation	31
4.6	Optimization Problem Formulation	32
4.7	Density Filtering and Projection	34
4.8	Algorithmic Workflow and Solver Implementation	36
5	Topology Optimization for Turbulent Flows: Results and Cross-Code CFD Verification	39
5.1	Introduction	39
5.2	Boundary Conditions and Evaluation Metrics	39
5.3	Benchmark 1: The Diffuser	40
5.4	Benchmark 2: The Double Pipe	41
5.5	Benchmark 3: The Pipe Bend	41
5.6	Benchmark 4: The U-Bend (Advanced Separation)	42
5.7	Evaluation of the Coupled "Semi-Frozen" Adjoint	42
6	Conclusion and Future Work	43
6.1	Conclusion	43
6.2	Limitations	44
6.3	Future Work	44
A	Configuration and Spatial Verification of the Baselines	46
A.1	Verification Benchmark 1: Channel Flow	46
A.2	Verification Benchmark 2: U-Bend Geometry	49
B	Mesh Independence of the Extracted Validation Geometries	53
B.1	Near-Wall Treatment and Meshing Strategy	53
B.2	Grid Convergence Analysis	54
C	Comparison of Optimized Designs with Laminar Baselines	56
C.1	Methodology and Evaluation Pipeline	56
C.2	Morphological and Hydrodynamic Comparison	57
C.3	Quantitative Performance Analysis	57
	Bibliography	59
	Code of Conduct	63

Abstract

As modern electronic and automotive systems grow increasingly compact and powerful, efficient thermal management has emerged as a fundamental engineering challenge. While topology optimization has proven to be a highly effective computational design tool for the fluid manifolds that drive these cooling systems, traditional methodologies rely heavily on laminar flow approximations. These laminar assumptions severely limit industrial applicability, as they fail to capture the separation, curvature effects, and turbulence-model dependence inherent to high-speed internal flows.

This thesis develops and implements a density-based topology optimization framework for turbulent internal flows as a baseline for future heat-transfer applications. The computational methodology is built upon the open-source FEniCS platform and utilizes the fictitious domain method with Brinkman penalization to conceptualize solid boundaries within a continuous fluid mesh. To capture the complex physics of high-Reynolds flows, the framework integrates the one-equation Spalart-Allmaras (SA) turbulence model. The optimization process is primarily driven by a frozen-turbulence continuous adjoint, while an exploratory semi-frozen coupled adjoint is also examined.

The assessment strategy is deliberately separated into three stages. First, the predictive scope of the SA model for curvature-dominated internal flows is established and benchmarked against Ansys Fluent. Second, the custom SA implementation in FEniCS is verified on turbulent benchmark flows. Third, the final optimized geometries are exported from FEniCS and re-analyzed in Ansys Fluent. This final stage compares FEniCS-SA with Ansys-SA on the same geometry and then evaluates the same designs with SST $k-\omega$ and, secondarily, standard $k-\epsilon$ to quantify turbulence-model sensitivity. In this revised framing, laminar-versus-turbulent design comparison is retained as supporting context rather than treated as CFD validation in itself.

The resulting thesis is therefore intentionally conservative in its claims. Its central question is not merely whether turbulent topologies look different from laminar ones, but whether SA-driven topology optimization produces low-loss designs whose predicted performance remains credible under independent CFD re-analysis. This establishes a more defensible and transparent baseline for future three-dimensional and conjugate heat-transfer extensions.

List of Figures and Tables

List of Figures

1.1	Iterative density-based topology optimization process: (a) initialization with a uniform field, (b) intermediate continuous distribution where $\gamma \in [0, 1]$, and (c) final discrete fluid topology where $\gamma \in \{0, 1\}$	2
1.2	Topology optimized manifolds with turbulent flow. Adapted from Dilgen et al. [15].	3
2.1	Streamwise velocity profiles in a 90° pipe bend ($R_c = 7D$, $Re_D \approx 34,000$, $De \approx 9,000$). The SA model performs relatively well, closely matching the experimental data and two-equation models for mild curvature. Reprinted from Apalowo [8].	10
2.2	Turbulence model predictions in a strongly curved internal U-duct ($R_c \approx 0.83H$, $Re_H = 100,000$, $De \approx 109,500$). Due to the extreme curvature, the standard Spalart-Allmaras model overpredicts eddy viscosity and fails to capture the inner-wall separation. Adapted from Rumsey et al. [32].	11
2.3	Comparison of velocity contours in a 180° U-bend ($R_c \approx 4.46D$, $Re_D \approx 44,700$, $De \approx 15,000$). The SA model performs well, closely matching the high-fidelity prediction.	12
2.4	Comparison of velocity contours in a 90° pipe bend ($R_c = 2.8D$, $Re_D \approx 43,000$, $De \approx 18,000$). The SA model over-smooths the velocity gradients compared to the $k - \omega$ SST model.	13
3.1	Quantitative code verification: line plots extracting data across the 2D periodic channel ($Re_H \approx 20,300$). The custom FEniCS SA formulation demonstrates excellent numerical agreement with the commercial Ansys Fluent SA baseline.	22
3.2	Code verification: qualitative velocity contours of the truncated 2D U-bend benchmark comparing the custom FEniCS implementation against Ansys Fluent. Both solvers utilize the Spalart-Allmaras model.	23
3.3	Quantitative validation: velocity profile across the U-bend exit ($R_c \approx 4.46D$, $Re_D \approx 44,700$, $De \approx 15,000$), comparing the custom FEniCS SA solver directly to the truncated Ansys Fluent baseline.	24

A.1	Wall-normal velocity profiles for the Coarse, Medium, and Fine grids in the periodic channel flow benchmark. The overlapping profiles demonstrate asymptotic spatial convergence.	47
A.2	Velocity profile comparison at the U-bend exit between the full Ansys domain (1.555 m legs) and the truncated topology optimization domain (30D legs). The complete overlap confirms the validity of the truncated geometry.	51
A.3	Velocity profiles at the U-bend exit for the Coarse, Medium, and Fine grids. The profiles demonstrate spatial convergence, justifying the selection of the Medium grid for the final verification.	52
B.1	Asymptotic convergence of the macroscopic pressure drop metric and localized velocity profiles across successive mesh refinement levels. . . .	55
C.1	Velocity magnitude contours of the laminar and turbulent designs, evaluated under identical high-Reynolds boundary conditions in Ansys Fluent (SST $k-\omega$).	57

List of Tables

A.1	Geometric, physical, and boundary-condition parameters for the periodic channel flow benchmark.	47
A.2	Numerical simulation parameters for the FEniCS steady Spalart-Allmaras channel benchmark.	48
A.3	Wall-resolved channel mesh hierarchy used for the grid-independence study. The first-layer height is kept fixed to maintain $y^+ \approx 1$, while the streamwise resolution, wall-normal resolution, and inflation-layer resolution are refined.	48
A.4	Geometric, physical, and boundary-condition parameters for the U-bend benchmark.	49
A.5	Numerical simulation parameters for the highly convective FEniCS U-bend benchmark.	50
A.6	Wall-resolved U-bend mesh hierarchy used for the grid-independence study. The first-layer height is kept fixed to maintain $y^+ \approx 1$, while the bulk element size and inflation-layer resolution are refined.	51
B.1	Grid convergence study for the extracted validation geometries. Metrics are reported for both the Ansys SA and SST $k-\omega$ models under identical boundary conditions.	54
C.1	Quantitative performance metrics of the laminar and turbulent optimized topologies evaluated under identical turbulent boundary conditions (Ansys SST $k-\omega$).	58

List of Abbreviations and Symbols

Abbreviations

CAD	Computer-Aided Design
CEV	Constant Eddy Viscosity
CFD	Computational Fluid Dynamics
CG1	Continuous Galerkin (Linear Basis Functions)
CHT	Conjugate Heat Transfer
DG0	Discontinuous Galerkin (Piecewise Constant Elements)
DNS	Direct Numerical Simulation
FEM	Finite Element Method
FVM	Finite Volume Method
IPCS	Incremental Pressure Correction Scheme
MMA	Method of Moving Asymptotes
PDE	Partial Differential Equation
RAMP	Rational Approximation of Material Properties
RANS	Reynolds-Averaged Navier-Stokes
SA	Spalart-Allmaras
SIMP	Solid Isotropic Material with Penalization
SST	Shear Stress Transport
SUPG	Streamline Upwind Petrov-Galerkin
TO	Topology Optimization
UFL	Unified Form Language

Symbols

Fluid Dynamics & Turbulence

d, y	Distance to the nearest wall / Hydraulic diameter [m]
De	Dean number [-]
k	Turbulent kinetic energy [m^2/s^2]
$p, \bar{p}, p'$	Instantaneous, mean, and fluctuating static pressure [Pa]
ΔP	Macroscopic total pressure drop [Pa]
R_c	Radius of curvature [m]
Re	Bulk Reynolds number [-]
$S, \tilde{S}$	Scalar vorticity magnitude and modified SA vorticity magnitude [$1/\text{s}$]
t	Time / Pseudo-time [s]
$\mathbf{u}, \bar{u}_i$	Velocity vector / Mean velocity component [m/s]
u'_i	Fluctuating velocity component [m/s]
u_τ	Friction velocity [m/s]
y^+	Non-dimensional wall distance [-]
μ, μ_t	Dynamic and turbulent eddy viscosity [$\text{Pa}\cdot\text{s}$]
ν, ν_t	Kinematic and turbulent eddy kinematic viscosity [m^2/s]
$\tilde{\nu}$	Spalart-Allmaras working variable [m^2/s]
ρ	Fluid density [kg/m^3]
Ω_{ij}	Vorticity tensor [$1/\text{s}$]
κ	von Kármán constant [-]

Numerical Implementation (FEniCS)

G, G_0	Relaxed Eikonal field variable and boundary value [$1/\text{m}$]
h, h_{avg}	Finite element cell diameter and average cell size [m]
$\ \cdot\ _{L_2}$	L_2 (Euclidean) norm [-]
$[[\cdot]]$	Jump operator across an internal element facet [-]
$\mathbf{n}$	Outward-pointing normal vector [-]
v, ξ	Test functions for pressure and turbulence equations [-]
$\alpha_u, \alpha_{\tilde{\nu}}$	Relaxation factors for velocity and turbulence [-]
Δt	Pseudo-time step [s]
ϵ_{tol}	Convergence tolerance [-]
ϕ	General scalar field variable [-]
σ_w	Eikonal relaxation constant [-]
τ	SUPG stabilization time parameter [s]

Topology Optimization

$\alpha(\gamma)$	Continuous Brinkman penalization term / inverse permeability
$\alpha_{solid}, \alpha_{fluid}$	Maximum solid and minimum fluid Brinkman penalties
α_{dg}	Artificial diffusion scaling factor for DG0 filter [-]
β	Heaviside projection steepness parameter [-]
γ, γ_{raw}	Continuous and raw design variable (1 for fluid, 0 for solid) [-]
$\tilde{\gamma}, \bar{\gamma}$	Filtered and Heaviside-projected design variables [-]
η	Heaviside projection threshold [-]
J	Objective function (Power Dissipation) [W]
q	RAMP penalization tuning parameter [-]
r	Continuous Helmholtz filter radius [m]
$\mathbf{s}, \mathbf{s}^*$	Monolithic forward state and continuous adjoint state vectors
V_f, V_{domain}	Physical fluid volume and total design domain volume [m ³]
V_{max}	Maximum allowable fluid volume fraction constraint [-]
$\Omega, \partial\Omega, \Gamma_{int}$	Computational domain, domain boundary, and internal element facets

Chapter 1

Introduction

1.1 Background and Motivation

With the continued miniaturization of components in the electronics and automotive industries, thermal management has become a critical bottleneck in the development of new technologies. While the heat sinks themselves are vital, the efficiency of any cooling system relies heavily on the fluid manifolds that deliver the coolant. If the flow distribution is uneven or if pumping the fluid requires excessive power due to massive pressure drops, the overall efficiency of the system severely degrades. Addressing this fluid delivery problem from an engineering perspective is a challenging task due to the complex aerodynamic behaviors that govern high-speed flow networks.

Innovative design strategies based on mathematical optimization have made it possible to improve the performance of these devices while capturing the complexity of the underlying physical phenomena. Among these strategies, topology optimization has emerged as a highly effective computational design tool for advanced thermal management [16]. For instance, the geometry of a complex microfluidic channel network can be automatically optimized to distribute coolant with specified flow rates without relying on human intuition [44]. Furthermore, the ultimate frontier of this technology lies in the design of conjugate and two-fluid heat exchangers, where the algorithm must simultaneously maximize heat transfer while strictly limiting the fluid pumping power [22]. However, before multi-physics heat transfer can be reliably optimized at an industrial scale, the fundamental algorithms governing the turbulent fluid dynamics must first be stabilized and validated.

1.2 Conceptual Overview of Density-Based Topology Optimization

To understand the challenges of applying optimization to turbulent fluids, it is first necessary to distinguish topology optimization from traditional design methodologies. Historically, computational design relied on size optimization (altering the thickness of predefined components) or shape optimization (moving the boundary

nodes of a predefined geometry). While effective, these methods are inherently limited by their starting topology. A shape optimization algorithm cannot spontaneously create a new hole or split a single fluid channel into two.

Density-based topology optimization, pioneered by Bendsøe and Kikuchi in 1988 for solid mechanics [11], approaches the problem fundamentally differently. Instead of moving boundaries, the design space (the computational domain) is discretized into a fixed grid of finite elements. The algorithm then determines the optimal distribution of material across this grid. In fluid problems, this is achieved by assigning a continuous design variable, $\gamma \in [0, 1]$, to every element in the domain. In the specific convention adopted for this thesis, a value of $\gamma = 1$ represents a pure fluid channel (e.g., the cooling air or water), while $\gamma = 0$ represents a solid, impermeable structure (e.g., a manifold wall).

However, because gradient-based optimizers require continuous and differentiable functions, this discrete binary choice is mathematically relaxed. The design variable is allowed to take intermediate "gray" values between 0 and 1. To give physical meaning to these intermediate states, the fluid equations are modified using a fictitious porous medium approach, where intermediate densities correspond to varying levels of flow permeability. The optimization algorithm iteratively updates the γ field (as conceptualized in Figure 1.1) to minimize a specific objective function (e.g., dissipated power) subject to physical constraints (e.g., a maximum allowable fluid volume fraction). The choice of the initial value for γ is a critical parameter for solver stability. A uniform initial guess of $\gamma = 0.5$ is standard practice to ensure an unbiased algorithmic search across the design space.

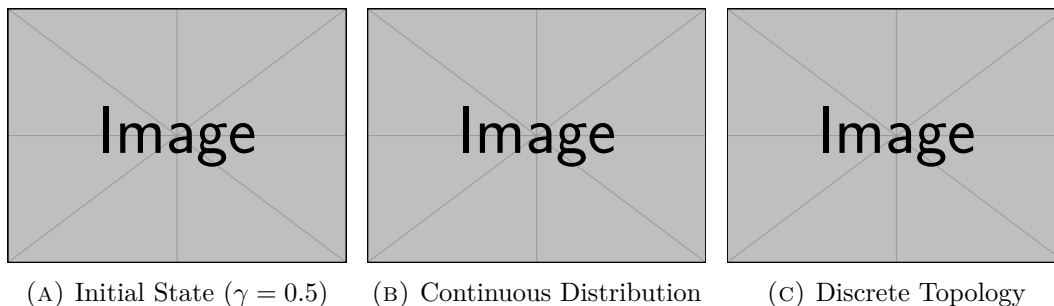


FIGURE 1.1: Iterative density-based topology optimization process: (a) initialization with a uniform field, (b) intermediate continuous distribution where $\gamma \in [0, 1]$, and (c) final discrete fluid topology where $\gamma \in \{0, 1\}$.

1.3 The Evolution of Fluid Topology Optimization

While density-based topology optimization became a standard tool in structural engineering in the late 1990s, its application to fluid dynamics is relatively recent. The optimization of fluid manifolds remains an active and highly challenging area of research [16]. The genesis of fluid topology optimization is widely attributed to Borrvall and Petersson (2003), who successfully applied the technique to mini-

mize power dissipation in creeping Stokes flow [12]. This was quickly followed by Gersborg-Hansen et al. (2005), who extended the framework to steady, laminar Navier-Stokes flows [18]. Over the past decade, the literature surrounding laminar flow optimization expanded extensively, successfully adapting the methodology to standard incompressible Navier-Stokes equations [14, 2, 44].

However, the assumption of laminar flow significantly simplifies the governing equations and the subsequent derivation of the adjoint sensitivities required for gradient-based optimization. Unfortunately, this assumption prevents the application of these methodologies to most industrial and high-performance applications, where flow velocities are high and turbulence dominates the flow physics. Only recently has the computational mechanics community begun to tackle the severe non-linearities and numerical instabilities associated with optimizing geometries directly in the fully turbulent regime [42, 15, 43].

Extending topology optimization to the turbulent regime is highly non-trivial. It requires coupling optimization algorithms with complex Reynolds-Averaged Navier-Stokes (RANS) turbulence models. These models introduce steep velocity gradients, highly non-linear behaviors, and severe numerical instabilities during the iterative optimization process. To mitigate this, gradient-based turbulent optimizations frequently rely on the "frozen turbulence" assumption, where the turbulent viscosity field is held constant during the sensitivity analysis to avoid differentiating the complex turbulence transport equations. While this assumption provides necessary solver stability and has driven massive success in industrial shape optimization for automotive aerodynamics [28], advancing toward fully coupled turbulent sensitivities in topology optimization remains an ongoing and highly complex mathematical challenge.

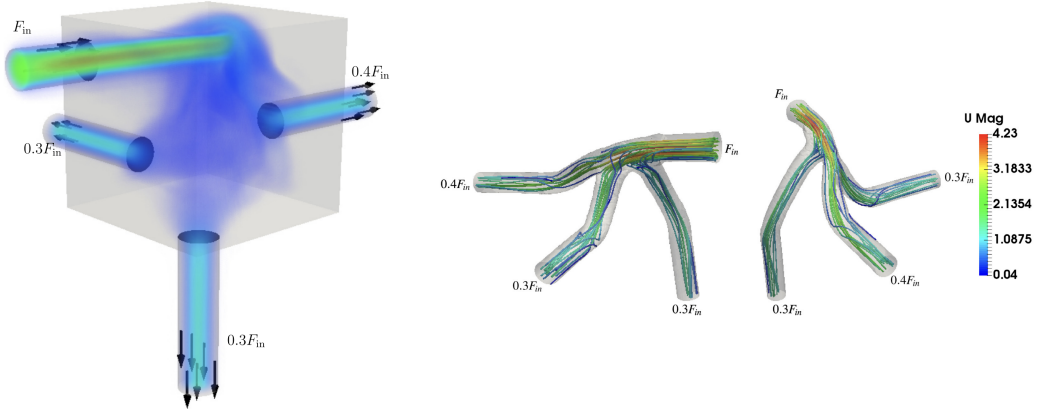


FIGURE 1.2: Topology optimized manifolds with turbulent flow. Adapted from Dilgen et al. [15].

Pioneering works by Yoon (2016) [42] and Dilgen et al. (2018) [15] have pushed the boundaries of these turbulent frameworks. As seen in Figure 1.2, integrating advanced turbulence models allows the optimizer to generate highly complex, organic fluid manifolds specifically tailored to manipulate and guide turbulent flow

distributions efficiently.

1.4 Objectives and Research Questions

The main objective of this Master’s thesis is to extend the application of topology optimization to the design of fluid manifolds operating strictly in the turbulent regime. To achieve this goal, this work investigates the coupling of fluid dynamics and turbulence modeling within an automated mathematical optimization framework. By strictly isolating the fluid mechanics from complex thermodynamic calculations, this thesis bounds its scope to establish a highly stable and mathematically rigorous baseline for pure aerodynamic manifold design.

Rather than relying solely on black-box commercial Computational Fluid Dynamics (CFD) software, the core of this thesis involves implementing the framework using **FEniCS**, an open-source computing platform for solving partial differential equations [3]. FEniCS, combined with high-level discrete adjoint methods (e.g., **dolfin-adjoint** [4]), provides the mathematical flexibility required to implement, verify, and integrate the one-equation Spalart-Allmaras (SA) turbulence model into a structural design loop. For full transparency and reproducibility, the complete FEniCS source code developed for this thesis is open-source and publicly available at: <https://github.com/TiebertLefebure/TurbulentTO.git>.

At this stage, it is important to distinguish two different comparisons that appear later in the thesis. A comparison between laminar-optimized and turbulent-optimized layouts is useful as supporting design context, because it shows how inertia and turbulence alter the preferred topology. However, this comparison is not treated as the principal CFD validation step. The decisive assessment instead consists of re-analyzing the final SA-optimized geometries in Ansys Fluent, first with the same SA model and then with alternative RANS closures.

Based on this methodology, this thesis seeks to answer the following specific research questions:

1. **How accurately does the Spalart-Allmaras turbulence model predict curvature-dominated internal turbulent flows, and where do its physical limitations appear?**
2. **Can a custom standard-Galerkin FEniCS implementation of the Spalart-Allmaras turbulence model converge robustly and achieve quantitative agreement with established commercial CFD solvers?**
3. **To what extent do SA-driven topology optimizations produce final designs whose predicted performance remains consistent under independent Ansys re-analysis, both with the same SA model and with alternative RANS closures?**

1.5 Outline of the Thesis

The remainder of this thesis is structured into the following five chapters:

- **Chapter 2 (Turbulence Modeling):** Provides the theoretical background on fluid dynamics and turbulence. It discusses the Navier-Stokes equations, the principles of RANS modeling, and the physical characteristics of the Spalart-Allmaras model.
- **Chapter 3 (Implementation and Verification of the Spalart-Allmaras Model in FEniCS):** Details the numerical implementation of the chosen turbulence model within the FEniCS framework, including fractional-step solvers, standard Galerkin discretization, convergence controls, and quantitative benchmark validations.
- **Chapter 4 (Topology Optimization for Turbulent Flows: Methodology):** Introduces the density-based optimization mathematical framework. It details the Brinkman penalization, the continuous adjoint derivations, the rationale for adopting the frozen turbulence assumption as the baseline design engine, and the exploratory coupled formulation.
- **Chapter 5 (Topology Optimization for Turbulent Flows: Results, Cross-Code Verification, and Model Sensitivity):** Presents the turbulent topology optimization results, documents the export of the final designs to Ansys Fluent, compares FEniCS-SA against Ansys-SA on the same geometries, and studies sensitivity to alternative turbulence models such as SST $k-\omega$ and standard $k-\epsilon$.
- **Chapter 6 (Conclusion and Future Work):** Summarizes the main findings, answers the research questions conservatively, and outlines the remaining limitations, with particular emphasis on turbulence-model dependence, geometry extraction, and future multi-physics heat-transfer applications.

Chapter 2

Turbulence Modeling

2.1 Introduction to Fluid Dynamics and Turbulence

Fluid dynamics is governed by the Navier-Stokes equations, which describe the conservation of mass and momentum for a viscous continuum fluid [40]. In this introductory text, these equations are presented using index notation (Einstein summation convention) to clearly illustrate the individual tensor components. In this convention, the indices i and j represent the three spatial dimensions (1, 2, and 3, corresponding to the x , y , and z axes). Consequently, x_i denotes the Cartesian coordinates, and u_i represents the corresponding components of the velocity vector. A repeated index within a single term implies a mathematical summation over all three spatial directions. (Note: In subsequent chapters dealing with finite element implementation, this notation will shift to standard vector calculus to align with FEniCS unified form language syntax).

The fundamental principle of mass conservation is expressed by the continuity equation. For a general, compressible fluid, this is defined as:

$$\frac{\partial \rho}{\partial t} + \frac{\partial(\rho u_i)}{\partial x_i} = 0 \quad (2.1)$$

where ρ is the fluid density. However, for most liquid cooling applications and low-speed fluid manifolds, the fluid can be safely assumed to be incompressible. Under the incompressible flow assumption, the density ρ is constant in both space and time. Therefore, the time derivative of density becomes zero, and the constant density can be factored out of the spatial derivative. Dividing the equation by ρ yields the simplified, incompressible continuity equation:

$$\frac{\partial u_i}{\partial x_i} = 0 \quad (2.2)$$

Similarly, for an incompressible, Newtonian fluid with a constant dynamic viscosity μ , the conservation of momentum is expressed as:

$$\rho \left(\frac{\partial u_i}{\partial t} + u_j \frac{\partial u_i}{\partial x_j} \right) = - \frac{\partial p}{\partial x_i} + \frac{\partial}{\partial x_j} \left(\mu \frac{\partial u_i}{\partial x_j} \right) \quad (2.3)$$

where p represents the static pressure of the fluid. In principle, solving Equation 2.3 directly yields the exact flow field. This approach is known as Direct Numerical Simulation (DNS). However, as the Reynolds number (Re) increases, the flow transitions into a chaotic turbulent state characterized by a broad spectrum of spatial and temporal scales [30]. Resolving the smallest turbulent eddies requires extremely fine spatial grids. For industrial applications, including the design of electronics cooling manifolds, DNS is computationally prohibitive. Therefore, engineers rely on turbulence modeling to predict the averaged effects of these fluctuations without resolving the complete turbulent spectrum [39, 5].

2.2 Reynolds-Averaged Navier-Stokes (RANS) Equations

The most common approach to turbulence modeling in engineering is the Reynolds-Averaged Navier-Stokes (RANS) framework, based on the theories originally introduced by Osborne Reynolds [30]. The core principle of RANS is the Reynolds decomposition. This mathematical operation separates the instantaneous flow variables into a time-averaged mean component ($\bar{u}_i, \bar{p}$) and a fluctuating component (u'_i, p'):

$$u_i = \bar{u}_i + u'_i \quad \text{and} \quad p = \bar{p} + p' \quad (2.4)$$

Substituting these decomposed variables into the standard Navier-Stokes equations and taking the time average yields the RANS momentum equations:

$$\rho \left(\frac{\partial \bar{u}_i}{\partial t} + \bar{u}_j \frac{\partial \bar{u}_i}{\partial x_j} \right) = -\frac{\partial \bar{p}}{\partial x_i} + \frac{\partial}{\partial x_j} \left(\mu \frac{\partial \bar{u}_i}{\partial x_j} - \rho \overline{u'_i u'_j} \right) \quad (2.5)$$

The averaging process introduces a new term, $-\rho \overline{u'_i u'_j}$, known as the Reynolds stress tensor. This tensor represents the momentum transfer caused by turbulent fluctuations. The introduction of this term creates a fundamental mathematical closure problem because the RANS equations now contain more unknowns than available physical equations [39].

2.2.1 The Boussinesq Hypothesis and the Isotropic Assumption

To close the system and solve for the mean flow, most RANS models rely on the Boussinesq eddy viscosity hypothesis [30]. This hypothesis assumes that the turbulent momentum transfer is analogous to molecular momentum transfer. It relates the Reynolds stresses linearly to the mean velocity gradients using a proportionality constant called the turbulent eddy viscosity (μ_t):

$$-\rho \overline{u'_i u'_j} = \mu_t \left(\frac{\partial \bar{u}_i}{\partial x_j} + \frac{\partial \bar{u}_j}{\partial x_i} \right) - \frac{2}{3} \rho k \delta_{ij} \quad (2.6)$$

where k is the turbulent kinetic energy and δ_{ij} is the Kronecker delta. The primary task of any Boussinesq-based turbulence model is to calculate this local eddy viscosity. It is crucial to highlight a fundamental physical limitation of Equation 2.6: the Boussinesq hypothesis inherently assumes that μ_t is an *isotropic* scalar quantity. It mathematically enforces that turbulence behaves identically in all spatial directions and aligns perfectly with the mean strain rate. While this assumption holds for simple shear flows, it systematically fails in predicting anisotropic phenomena, such as the strong secondary flows induced by streamline curvature.

2.3 Boundary Layer Theory and Near-Wall Treatment

Before selecting a specific turbulence equation to calculate μ_t , the physical structure of a turbulent boundary layer must be considered, as the model must accurately resolve these near-wall regions. The boundary layer is traditionally divided into three distinct regions based on the non-dimensional wall distance (y^+) and non-dimensional velocity (u^+):

$$y^+ = \frac{yu_\tau}{\nu}, \quad u^+ = \frac{\bar{u}}{u_\tau} \quad (2.7)$$

where y is the absolute distance from the wall, ν is the kinematic viscosity (μ/ρ), and $u_\tau = \sqrt{\tau_w/\rho}$ is the friction velocity derived from the wall shear stress (τ_w) [30].

1. **The Viscous Sublayer** ($y^+ < 5$): Immediately adjacent to the wall, turbulent fluctuations are damped out by kinematic constraints. Viscous shear dominates, and the velocity profile is strictly linear ($u^+ = y^+$).
2. **The Buffer Layer** ($5 < y^+ < 30$): A transition region where both viscous and turbulent shear stresses are of equal magnitude.
3. **The Log-Law Region** ($y^+ > 30$): Farther from the wall, turbulence dominates, and the velocity follows a logarithmic distribution predicted by the von Kármán constant [39].

High-Reynolds models (like the standard $k - \epsilon$ model) use empirical "wall functions" to bridge the gap between the wall and the log-law region, requiring the first computational mesh element to be placed at $y^+ > 30$. However, for topology optimization, where solid-fluid boundaries continuously evolve during the optimization loop, empirical wall functions introduce severe mathematical discontinuities and numerical instabilities. Therefore, a "wall-resolved" model that integrates the governing equations all the way through the viscous sublayer ($y^+ \approx 1$) is strictly required for robust gradient-based optimization.

2.4 The Spalart-Allmaras One-Equation Model

While two-equation models (such as $k - \epsilon$ or $k - \omega$) solve separate transport equations for the turbulent kinetic energy and a turbulent length scale, they introduce highly

non-linear source terms. These terms frequently cause numerical instabilities during the adjoint sensitivity analysis required for topology optimization. To achieve a trade-off between physical accuracy and computational efficiency, this thesis utilizes the Spalart-Allmaras (SA) model, originally developed in 1992 for aerodynamic flows with adverse pressure gradients [34].

The SA model is a one-equation model that solves a single transport equation directly for a working variable, $\tilde{\nu}$, which is subsequently linked to the true kinematic eddy viscosity ($\nu_t = \mu_t/\rho$) [35]. The transport equation for the standard SA model is given by:

$$\frac{\partial \tilde{\nu}}{\partial t} + \bar{u}_j \frac{\partial \tilde{\nu}}{\partial x_j} = c_{b1} \tilde{S} \tilde{\nu} - c_{w1} f_w \left(\frac{\tilde{\nu}}{d} \right)^2 + \frac{1}{\sigma} \left[\frac{\partial}{\partial x_j} \left((\nu + \tilde{\nu}) \frac{\partial \tilde{\nu}}{\partial x_j} \right) + c_{b2} \left(\frac{\partial \tilde{\nu}}{\partial x_j} \right)^2 \right] \quad (2.8)$$

The terms on the right-hand side represent the production, destruction, and diffusion of the turbulent working variable, respectively. Crucially, the production term relies on a modified vorticity scalar, $\tilde{S}$. This indicates that turbulence generation in the SA model is fundamentally driven by fluid shear, a physical characteristic that introduces unique boundary-condition sensitivities during advanced adjoint calculations (discussed further in Chapter 4).

Furthermore, the destruction term is explicitly a function of d , the absolute distance to the nearest wall. This makes the SA model a highly effective "wall-resolved" low-Reynolds model. It is inherently sensitive to near-wall damping effects and integrates smoothly through the viscous sublayer without requiring complex empirical wall functions or a second length-scale equation [31]. Detailed derivations of the model constants and wall damping functions are deferred to Chapter 3, where the numerical implementation in FEniCS is discussed.

2.5 Evaluation of Curvature Effects and the Dean Number

Despite its computational advantages for topology optimization, the standard SA model relies on a single length scale tied to the nearest wall distance, making it mathematically blind to the anisotropic physics of streamline curvature. When fluid passes through a bend, centrifugal forces induce secondary flows perpendicular to the main flow direction. To quantify the strength of these secondary flows, the dimensionless Dean number (De) is utilized:

$$De = Re_d \sqrt{\frac{d}{2R_c}} \quad (2.9)$$

where d is the characteristic length of the cross-section (e.g., the pipe diameter D or hydraulic diameter D_h), R_c is the radius of curvature, and Re_d is the bulk Reynolds number based strictly on that exact same characteristic length. A higher Dean number indicates a stronger cross-sectional influence of centrifugal forces.

A review of the literature demonstrates that the predictive capability of the SA model is heavily dependent on this Dean number. For mildly curved flows, the SA model performs relatively well. Apalowo [8] evaluated a 90° circular pipe bend with a Dean number of $De \approx 9,000$ (operating at $Re_D \approx 34,000$, based on the pipe diameter). Under these moderate conditions, where the bend radius was seven times larger than the pipe diameter ($R_c = 7D$), the SA model provided a highly acceptable approximation of the streamwise velocity profiles compared to higher-order models, successfully capturing the primary flow physics.

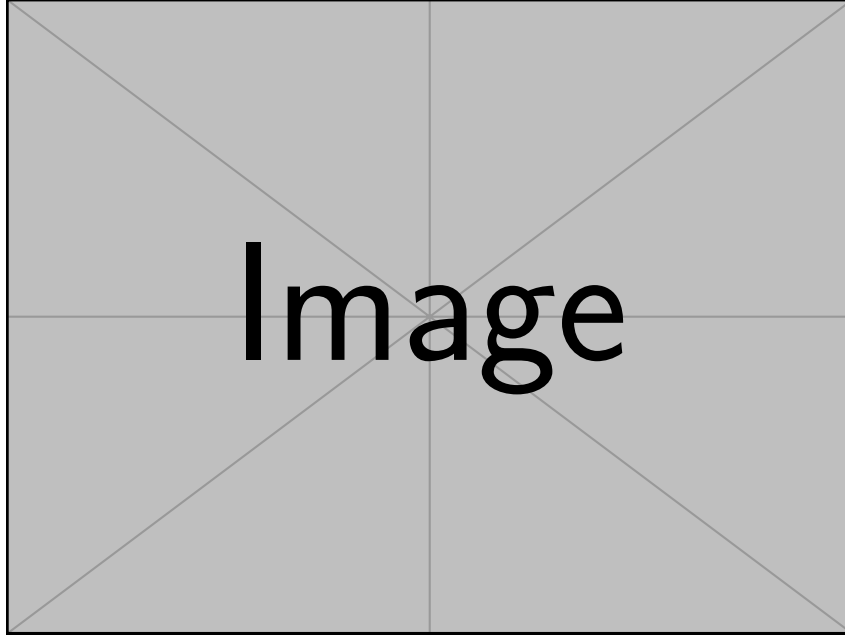


FIGURE 2.1: Streamwise velocity profiles in a 90° pipe bend ($R_c = 7D$, $Re_D \approx 34,000$, $De \approx 9,000$). The SA model performs relatively well, closely matching the experimental data and two-equation models for mild curvature. Reprinted from Apalowo [8].

However, as the curvature tightens and the Dean number increases, the limitations of the isotropic Boussinesq assumption become severe. Rumsey et al. [32] evaluated the internal flow through a strongly curved 180° turnaround duct (the Monson U-Duct). While this benchmark operated at a higher bulk Reynolds number ($Re_H = 100,000$, based on the duct height H), the extremely tight geometry ($R_c \approx 0.83H$) generated massive centrifugal forces and an extreme equivalent Dean number.

It is critical to note that the Spalart-Allmaras model was fundamentally designed for high-Reynolds aerospace applications and remains mathematically stable well beyond $Re = 100,000$ in straight or mildly curved flows. Therefore, the subsequent failure of the model in this benchmark cannot be attributed to inertial scaling, but is strictly driven by the extreme geometric curvature. Under these severe centrifugal conditions, the uncorrected SA model drastically overpredicted the production of

eddy viscosity on the inner radius of the bend. This artificial excess of viscosity suppressed the adverse pressure gradients, causing the standard SA formulation to completely fail at predicting the massive flow separation bubble that physically occurs on the inner convex wall (Figure 2.2).

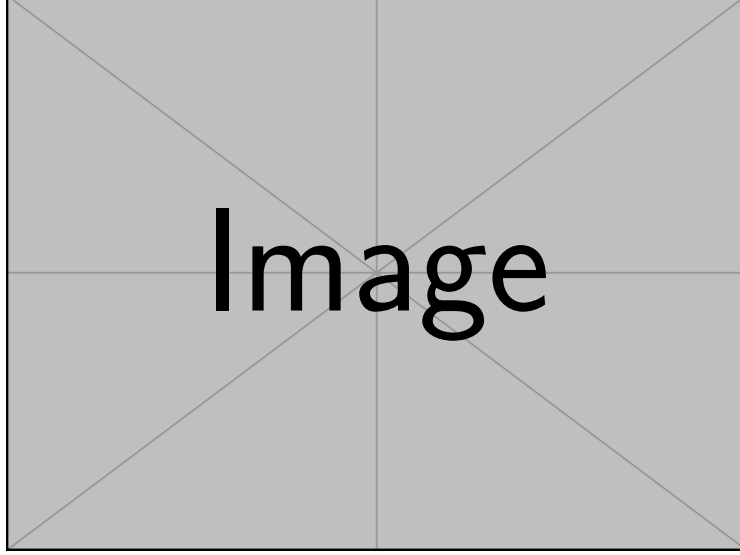


FIGURE 2.2: Turbulence model predictions in a strongly curved internal U-duct ($R_c \approx 0.83H$, $Re_H = 100,000$, $De \approx 109,500$). Due to the extreme curvature, the standard Spalart-Allmaras model overpredicts eddy viscosity and fails to capture the inner-wall separation. Adapted from Rumsey et al. [32].

While the literature clearly establishes that the SA model is viable at $De \approx 9,000$ and fails catastrophically in extreme bends, the specific operational threshold between these extremes must be defined to justify its use in topology optimization. Therefore, to explicitly isolate the threshold where the model breaks down, rigorous benchmark studies were subsequently conducted within Ansys Fluent.

2.6 Benchmark Studies in Ansys Fluent

To investigate the exact physical breaking point of the SA model, benchmark simulations were performed using **Ansys Fluent**. The study focused on two distinct, fully three-dimensional (3D) internal flow geometries: a 180° U-bend and a tighter 90° pipe bend. To ensure a standardized and rigorously tested numerical baseline, the computational meshes, fluid properties, and boundary conditions for these simulations were adopted directly from the official Ansys Fluid Dynamics Verification Manual [7]. Specifically, the 90° bend corresponds to verification case VMFL030, while the 3D U-bend corresponds to case VMFL048.

These cases evaluate industrial flow regimes ($Re_D > 40,000$) to ensure the turbulence model is rigorously verified against dominant inertial physics before being implemented into the custom FEniCS solver. To evaluate the accuracy of the one-

equation model, the results were compared directly against the $k - \omega$ SST model, an industry standard for accurately predicting adverse pressure gradients and flow separation.

2.6.1 Mild Curvature: The U-Bend Case

The first benchmark evaluated a 3D U-bend geometry characterized by a moderate curvature ratio of $R_c \approx 4.46D$. Operating at a bulk Reynolds number of $Re_D \approx 44,700$, this geometry resulted in a Dean number of $De \approx 15,000$. Consistent with the findings of Apalowo for moderate bends, the SA model performed well under these conditions. As shown in Figure 2.3(b), the velocity contours generated by the SA model closely matched those of the high-fidelity $k - \omega$ SST model (Figure 2.3(a)), accurately capturing the bulk flow distribution without severe artificial diffusion.

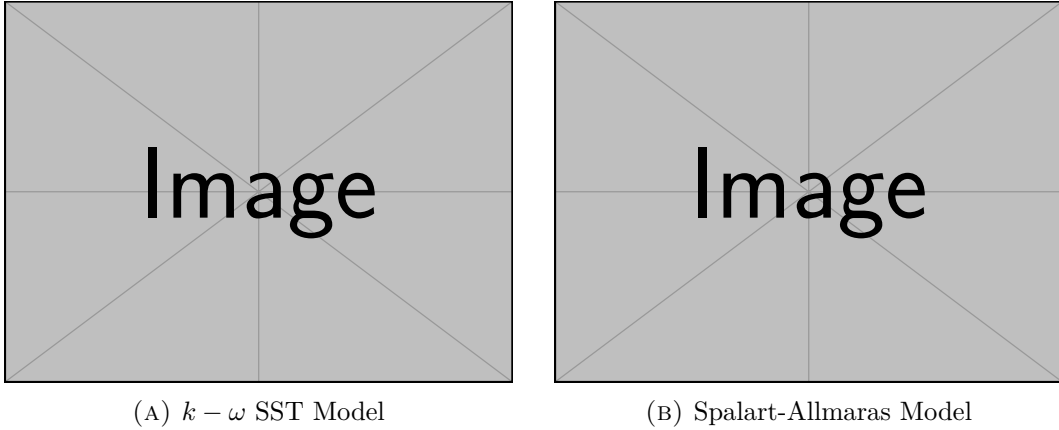


FIGURE 2.3: Comparison of velocity contours in a 180° U-bend ($R_c \approx 4.46D$, $Re_D \approx 44,700$, $De \approx 15,000$). The SA model performs well, closely matching the high-fidelity prediction.

2.6.2 Strong Curvature: The 90° Bend Case

The second benchmark evaluated a tighter 90° pipe bend characterized by a severe curvature ratio of $R_c = 2.8D$. Despite operating at a very similar bulk Reynolds number of $Re_D \approx 43,000$ compared to the U-bend, this significantly tighter bend radius pushed the Dean number up to $De \approx 18,000$. Because the bulk flow speed and Reynolds number remain effectively constant between these two benchmark cases, this comparison successfully isolates the geometric effect of streamline curvature. It clearly demonstrates that the subsequent failure of the SA model is driven purely by the tighter bend radius, which elevates the Dean number and amplifies anisotropic centrifugal effects.

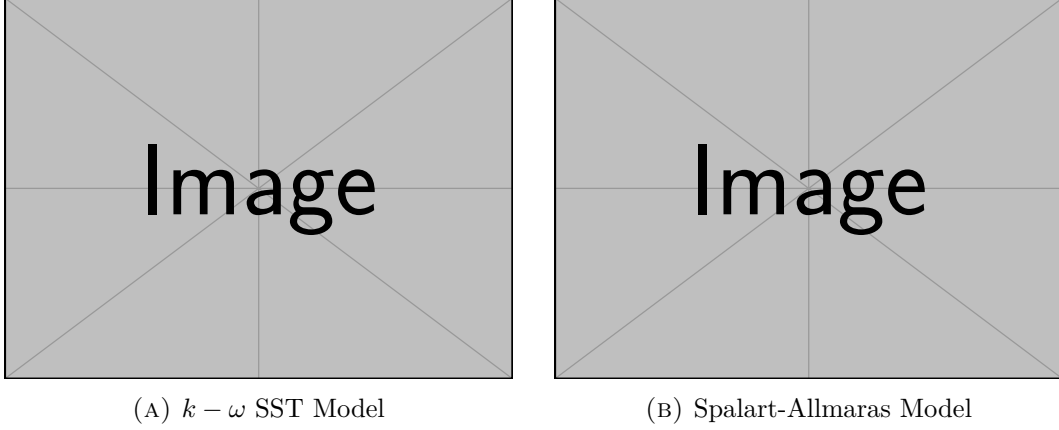


FIGURE 2.4: Comparison of velocity contours in a 90° pipe bend ($R_c = 2.8D$, $Re_D \approx 43,000$, $De \approx 18,000$). The SA model over-smooths the velocity gradients compared to the $k - \omega$ SST model.

2.7 Conclusion

This chapter directly addresses the first research question regarding the predictive capabilities and physical limitations of the Spalart-Allmaras model. The theoretical breakdown and Ansys benchmark studies confirm that for internal geometries with moderate curvature (up to $De \approx 15,000$), the SA model performs accurately and successfully captures the primary flow physics. Furthermore, because it is a wall-resolved model, it avoids the numerical pitfalls of empirical wall functions, making it exceptionally well-suited for boundary-evolving topology optimization.

However, its physical limitation lies in the isotropic Boussinesq assumption. When streamline curvature induces strong anisotropic secondary flows (heuristically observed to cause model breakdown beyond a threshold of $De \approx 18,000$), the uncompensated SA model systematically overpredicts eddy viscosity and fails to capture flow separation. Consequently, while the SA model provides the necessary numerical stability to drive the optimization algorithms in FEniCS, the final topological designs must be critically verified. Any complex manifold generated by the SA optimizer must be subsequently evaluated in commercial software using a higher-fidelity two-equation model to confirm its true aerodynamic performance.

Chapter 3

Implementation and Verification of the Spalart-Allmaras Model in FEniCS

3.1 Introduction to FEniCS and the Finite Element Method

With the theoretical foundation of the Spalart-Allmaras (SA) model established in Chapter 2, this chapter details its numerical translation into a working computational solver. The numerical simulations and eventual topology optimization in this thesis are built upon FEniCS, an open-source computing platform for solving partial differential equations (PDEs). Unlike commercial computational fluid dynamics (CFD) software such as Ansys, which typically relies on the Finite Volume Method (FVM) and provides a "black box" graphical interface, FEniCS uses the Finite Element Method (FEM) and requires the user to explicitly define the mathematical formulation of the problem [25].

To solve a PDE in FEniCS, the governing equations (the "strong form") must be mathematically transformed into a "weak formulation" (or variational form). FEniCS utilizes the Unified Form Language (UFL) to express these weak forms in Python code that closely mirrors the mathematical notation [23]. This high-level mathematical scripting is particularly advantageous for topology optimization, as it allows for the automated derivation of the adjoint equations required for gradient-based optimization algorithms.

To ensure complete transparency and reproducibility, the entire custom codebase developed for this thesis is available in the public GitHub repository `TurbulentT0`. To maintain modularity, the repository is divided into two primary directories. The `TurbulenceModels` directory contains the standalone Spalart-Allmaras fluid dynamics engine and the verification benchmarks discussed throughout this chapter. Conversely, the `FluidT0` directory houses the optimization algorithms, penalty formulations, and objective functions that will be detailed in Chapters 4 and 5.

3.1.1 Derivation of a Weak Formulation

Before diving into the complex turbulence equations, it is instructive to demonstrate how a continuous PDE is translated into a discrete FEniCS formulation. To illustrate this methodology, consider the Poisson equation for pressure, which is a fundamental component of incompressible fluid solvers. The strong form of the Poisson equation is given by:

$$-\nabla^2 p = f \quad \text{in } \Omega \quad (3.1)$$

where p is the pressure field, f is a source term (derived from the velocity divergence), and Ω is the computational domain. To obtain the weak form, the equation is multiplied by a sufficiently smooth test function v and integrated over the entire domain Ω , following standard Galerkin finite element procedures [25, 23]:

$$-\int_{\Omega} (\nabla^2 p) v \, dx = \int_{\Omega} f v \, dx \quad (3.2)$$

Applying integration by parts (Green's first identity) to the left-hand side yields:

$$\int_{\Omega} \nabla p \cdot \nabla v \, dx - \int_{\partial\Omega} (\nabla p \cdot \mathbf{n}) v \, ds = \int_{\Omega} f v \, dx \quad (3.3)$$

where $\partial\Omega$ is the boundary of the domain and $\mathbf{n}$ is the outward-pointing normal vector. This integral formulation fundamentally lowers the continuity requirements of the solution, requiring only first-order derivatives to be square-integrable. Furthermore, Neumann boundary conditions (e.g., a prescribed pressure gradient at an outlet) can be naturally substituted directly into the boundary integral $\int_{\partial\Omega} (\nabla p \cdot \mathbf{n}) v \, ds$. This variational approach forms the foundation of all fluid and turbulence equations implemented in this thesis.

3.2 Numerical Solution Strategy for Fluid Dynamics

Solving the full Reynolds-Averaged Navier-Stokes (RANS) equations is numerically challenging due to the non-linear convective terms, the saddle-point nature of the coupled velocity-pressure system, and the intense cross-coupling with the turbulence model. To achieve a stable and efficient steady-state solution in FEniCS that seamlessly translates into the topology optimization framework, the governing equations are rigorously decoupled.

3.2.1 Decoupling the Physics: The Picard Iteration Scheme

Because the momentum equation relies on the turbulent eddy viscosity (ν_t), and the turbulence transport equation relies on the convective velocity ($\mathbf{u}$), solving the entire fluid-turbulence system simultaneously in a massive monolithic matrix frequently leads to severe ill-conditioning and solver divergence.

To resolve this, the forward physics are decoupled using a **Picard iteration scheme** (successive substitution), a highly robust approach for segregated RANS

solvers [17, 39]. During each Picard step, the flow solver advances using the eddy viscosity from the previous iteration. The newly updated velocity field is then passed to the Spalart-Allmaras solver to calculate a new eddy viscosity field. This alternating sequence forms the outer loop of the solver, iteratively updating the decoupled equations until the fully coupled physical system reaches a steady-state equilibrium.

3.2.2 Incremental Pressure Correction Scheme (IPCS)

Within the Picard loop, the Navier-Stokes equations themselves must be solved. Rather than solving for velocity and pressure simultaneously, this implementation utilizes an Incremental Pressure Correction Scheme (IPCS). This algorithmic splitting is fundamentally based on the fractional-step projection methods originally pioneered by Chorin [13] and Temam [37]. To nurse the non-linear convective momentum terms to a stable steady state, a pseudo-time stepping approach (Δt) is employed strictly for the flow variables. The IPCS algorithm splits the fluid equations into three sequential steps per pseudo-time step:

1. **Tentative Velocity Step:** The momentum equation is solved using the pressure field from the previous iteration (p^n) to compute an intermediate, non-divergence-free velocity field ($\mathbf{u}^*$). Because this is a RANS framework, the diffusion term utilizes the effective kinematic viscosity, $(\nu + \nu_t)$. Treating the convective term explicitly and the diffusive term implicitly, the semi-discrete equation is:

$$\frac{\mathbf{u}^* - \mathbf{u}^n}{\Delta t} + (\mathbf{u}^n \cdot \nabla) \mathbf{u}^* - \nabla \cdot ((\nu + \nu_t^n) \nabla \mathbf{u}^*) = -\nabla p^n \quad (3.4)$$

2. **Pressure Correction Step:** A Poisson equation is solved to find the pressure variation that enforces the continuity constraint ($\nabla \cdot \mathbf{u}^{n+1} = 0$). The equation for the new pressure field (p^{n+1}) is derived as:

$$\nabla^2 p^{n+1} = \nabla^2 p^n + \frac{1}{\Delta t} \nabla \cdot \mathbf{u}^* \quad (3.5)$$

3. **Velocity Update Step:** The tentative velocity is strictly corrected using the gradient of the pressure difference to yield a mass-conserving velocity field ($\mathbf{u}^{n+1}$) for the current iteration:

$$\mathbf{u}^{n+1} = \mathbf{u}^* - \Delta t \nabla (p^{n+1} - p^n) \quad (3.6)$$

The simulation is deemed fully converged to a steady state when the L_2 norm of the residual difference between successive Picard iterations falls below a strictly defined tolerance limit (ϵ_{tol}) for both the fluid and turbulence fields.

3.3 Detailed Formulation of the Spalart-Allmaras Model

As concluded in Chapter 2, the Spalart-Allmaras (SA) one-equation model was selected for its balance of physical accuracy and numerical stability. Unlike the transient flow equations, the SA transport equation is solved in its purely steady-state form within the Picard loop. The steady transport equation for the working variable $\tilde{\nu}$ is given by:

$$\mathbf{u} \cdot \nabla \tilde{\nu} = c_{b1} \tilde{S} \tilde{\nu} - c_{w1} f_w \left(\frac{\tilde{\nu}}{y} \right)^2 + \frac{1}{\sigma} [\nabla \cdot ((\nu + \tilde{\nu}) \nabla \tilde{\nu}) + c_{b2} (\nabla \tilde{\nu})^2] \quad (3.7)$$

where ν is the molecular kinematic viscosity, $\sigma = 2/3$ is the turbulent Prandtl number, c_{b1} and c_{w1} are calibrated closure constants, $\tilde{S}$ is a modified vorticity magnitude, f_w is the wall-damping function, and y is the physical distance to the nearest solid boundary [35].

3.3.1 Closure Terms and Calibration Constants

The Spalart-Allmaras model relies on several interconnected closure functions to accurately capture boundary layer physics. The production term relies on a modified vorticity magnitude, $\tilde{S}$, which prevents the destruction of turbulence in regions where the vorticity goes to zero, while maintaining correct scaling near the wall [35]. It is defined as:

$$\tilde{S} = S + \frac{\tilde{\nu}}{\kappa^2 y^2} f_{v2} \quad (3.8)$$

where $S = \sqrt{2\Omega_{ij}\Omega_{ij}}$ is the scalar magnitude of the vorticity tensor, κ is the von Kármán constant, and y is the distance to the nearest wall. The function f_{v2} is a near-wall modifier:

$$f_{v2} = 1 - \frac{\chi}{1 + \chi f_{v1}} \quad \text{where} \quad \chi = \frac{\tilde{\nu}}{\nu} \quad \text{and} \quad f_{v1} = \frac{\chi^3}{\chi^3 + c_{v1}^3} \quad (3.9)$$

Similarly, the wall destruction term is modulated by the non-dimensional function f_w , which accelerates the decay of turbulent viscosity inside the viscous sublayer:

$$f_w = g \left(\frac{1 + c_{w3}^6}{g^6 + c_{w3}^6} \right)^{1/6} \quad (3.10)$$

where the intermediate variables g and r are evaluated as:

$$g = r + c_{w2}(r^6 - r) \quad \text{and} \quad r = \min \left(\frac{\tilde{\nu}}{\tilde{S} \kappa^2 y^2}, 10 \right) \quad (3.11)$$

The upper bound of 10 on r ensures numerical stability during solver initialization when $\tilde{S}$ may momentarily approach zero.

It should be explicitly noted that the original formulation by Spalart and Allmaras [35] includes additional geometric trip functions (f_{t1} and f_{t2}) designed to manually trigger the transition from laminar to turbulent flow at specific user-defined coordinates. However, following the topology optimization methodology established by Yoon [42], these transition functions are deliberately omitted from this thesis. Because the topological boundaries are unknown *a priori* and evolve during optimization, tracking discrete trip points is mathematically unfeasible; therefore, the model assumes a fully turbulent flow regime.

To close the system, the standard empirical calibration constants formulated by Spalart and Allmaras are utilized throughout the FEniCS implementation [31]:

$$\begin{aligned} c_{b1} &= 0.1355 & \sigma &= 2/3 & c_{b2} &= 0.622 \\ \kappa &= 0.4187 & c_{w2} &= 0.3 & c_{w3} &= 2.0 \\ c_{v1} &= 7.1 & c_{w1} &= \frac{c_{b1}}{\kappa^2} + \frac{1 + c_{b2}}{\sigma} \end{aligned}$$

3.3.2 Wall-Distance Calculation via the Relaxed Eikonal Equation

A major computational challenge in the SA model is the calculation of the wall distance y . In a standard static CFD simulation, y is purely geometric and can be pre-computed. However, to ensure this verification solver translates seamlessly into the topology optimization code (where solid boundaries evolve continuously), the implementation abandons discrete geometric searches in favor of a PDE-based approach.

The true geometric wall distance satisfies the standard Eikonal equation, $|\nabla y| = 1$. However, this formulation contains singularities at the medial axes of the domain. To guarantee continuous differentiability—a strict requirement for later adjoint derivations—a relaxed Eikonal equation proposed by Yoon [42] is solved:

$$\nabla G \cdot \nabla G + \sigma_w G (\nabla \cdot \nabla G) = (1 + 2\sigma_w) G^4 \quad (3.12)$$

Here, G is an intermediate, non-dimensional field variable. The parameter σ_w is a relaxation constant (typically set to 0.1) that controls the smoothness of the field. The PDE is subjected to a Dirichlet boundary condition at the physical walls, $G = G_0$, where G_0 is an arbitrarily large constant (e.g., $G_0 = 20.0$). Once the field G is solved over the domain, the physical wall distance y is recovered algebraically at every node using the inverse relationship:

$$y = \frac{1}{G} - \frac{1}{G_0} \quad (3.13)$$

This system is implemented in the core solver suite within the `Utilities.py` script. As shown in Listing 3.1, once the non-linear variational problem for G is solved, the physical distance y is recovered and strictly bounded from below to prevent numerical singularities.

LISTING 3.1: Implementation of Yoon’s relaxed Eikonal equation for wall-distance calculation, extracted from `Utilities.py`.

```

# Weak form corresponding to Yoon 2016 Eq. (19):
F = (
    (Constant(1.0) - sigma_w_const) * inner(grad(G), grad(G)) * z
    * dx
    - sigma_w_const * G * inner(grad(G), grad(z)) * dx
    - (Constant(1.0) + Constant(2.0) * sigma_w_const) * G**4 * z *
    dx
)
problem = NonlinearVariationalProblem(F, G, bcs=wall_bcs, J=
    derivative(F, G))
solver = NonlinearVariationalSolver(problem)
solver.solve()

# Bound G to prevent division by zero
G.vector()[:] = np.maximum(G.vector()[:], float(g_floor))

# Recover physical wall distance (y)
y = project(Constant(1.0) / G - Constant(1.0) / g0_const,
    Space)
return bound_from_bellow(y, 0.0)

```

3.4 Variational Formulation and FEniCS Implementation

The standard Galerkin weak formulation is constructed by multiplying the steady transport Equation 3.7 by a test function ξ and applying integration by parts to the diffusive terms. Because the equation is highly non-linear, robust convergence is heavily dependent on the quality of the mesh and the stability of the external velocity field provided by the Picard outer loop.

The core SA physics, boundary conditions, continuous Eikonal formulations, and the overarching Picard iteration scheme were derived independently for this thesis. However, the underlying FEniCS solver architecture was structurally inspired by an open-source $k - \epsilon$ repository developed by Marcibál [26, 21], specifically regarding the procedural execution of the variational forms and the implementation of the Incremental Pressure Correction Scheme (IPCS). The explicit translation of the SA mathematical formulation is handled by the `SpalartAllmarasSteady` class within the `TurbulenceModel_SpalartAllmaras.py` script. As demonstrated in Listing 3.2, the Unified Form Language (UFL) mirrors the analytical weak form precisely, utilizing the most recent convective velocity field (`external_velocity`) passed from the IPCS flow solver.

LISTING 3.2: FEniCS implementation of the steady-state SA standard Galerkin weak formulation. Python dictionary lookups have been simplified for readability.

```

def construct_forms(self, external_velocity):
    self._construct_turbulent_quantities(external_velocity)
    sigma = Constant(2.0 / 3.0)

    # Steady-state SA transport equation weak form

```

```

FNT = (
    dot(external_velocity, nabla_grad(self._nu_tilde)) * self._xi
    * self._dx
    + inner((self._nu_laminar + self._nu_tilde0) / sigma * grad(
        self._nu_tilde), grad(self._xi)) * self._dx
    + self._react_nt * self._nu_tilde * self._xi * self._dx
    - self._source_nt * self._xi * self._dx
)

self._a_nt = lhs(FNT)
self._l_nt = rhs(FNT)

```

3.5 General Solver Configuration and Numerical Parameters

Before detailing the specific verification benchmarks, it is necessary to establish the general numerical configuration of the FEniCS solver. Both the channel flow and the U-bend verifications were simulated using the segregated algorithmic structure detailed earlier: an outer Picard fixed-point loop couples the fluid momentum equations to the Spalart-Allmaras transport equation, while an inner IPCS resolves the velocity-pressure subsystem via pseudo-time stepping.

Because the U-bend domain introduces severe geometric curvature, strong convective transport, and adverse pressure gradients, it represents a significantly more demanding numerical challenge than a fully developed, unidirectional channel flow. Consequently, the U-bend simulations necessitated stricter numerical controls to maintain stability, including a heavily refined IPCS pseudo-time step ($\Delta t = 1.0 \times 10^{-4}$ s) and much tighter under-relaxation of the turbulence field updates ($\alpha_{\tilde{\nu}} = 0.01$). To ensure strict scientific reproducibility without disrupting the narrative flow of this verification, the comprehensive matrices detailing all physical boundary conditions, geometric dimensions, and exact numerical solver parameters for both benchmarks have been compiled and are documented extensively in Sections A.1 and A.2 of Appendix A.

3.6 Verification Benchmark 1: Channel Flow

To verify the fundamental correctness of the custom fluid and turbulence solvers, the code was first tested on a standard 2D turbulent channel flow. To cleanly isolate and verify turbulence production, wall damping, and Picard coupling without the artifacts of a developing boundary layer, this benchmark was strictly formulated as a pressure-driven periodic channel rather than a conventional inlet-outlet duct.

The velocity field ($\mathbf{u}$) and Spalart-Allmaras working variable ($\tilde{\nu}$) were constrained periodically between the streamwise boundaries, while the top and bottom boundaries were treated as standard no-slip walls ($\mathbf{u} = 0, \tilde{\nu} = 0$). Flow was driven entirely by a fixed pressure drop from the upstream plane ($p = 2$ Pa) to the downstream

plane ($p = 0$ Pa). Based on a reference bulk velocity of $U_{\text{ref}} = 18.5$ m/s and utilizing the full channel height ($H = 2.0$ m) as the characteristic length scale, the solver was targeted to resolve a fully turbulent flow regime at a channel Reynolds number of $Re_H \approx 20,300$. By explicitly defining the Reynolds number based on the channel height rather than the mathematical hydraulic diameter, this formulation directly aligns with standard academic conventions for fundamental 2D channel flow verification. To rigorously verify the mathematical accuracy of the formulation, the FEniCS results were compared directly against the commercial Ansys Fluent solver operating under identical periodic boundary conditions.

3.6.1 Mesh Resolution Requirements

Because the SA formulation is a "low-Reynolds" model that does not utilize empirical wall functions, it requires mathematical integration directly through the viscous sublayer. Consequently, standard uniform computational grids are physically insufficient. To satisfy this strict boundary layer requirement, a custom structured inflation mesh was generated for the channel flow. Because the domain is a simple periodic rectangle, this highly anisotropic grid was constructed programmatically utilizing NumPy arrays and converted into a FEniCS-compatible format via the open-source `meshio` library. Based on the target bulk Reynolds number, this approach explicitly enforced a physical first-cell height of approximately 1.7×10^{-3} meters, guaranteeing a non-dimensional wall distance of $y^+ \approx 1$ across the entire domain.

Furthermore, to guarantee that the numerical results were not influenced by spatial discretization, a formal grid independence study was conducted. The full results of this spatial verification, proving convergence, are documented in Appendix [A.1.3](#).

3.6.2 FEniCS vs. Ansys Comparison

The channel flow was simulated in FEniCS using the decoupled Picard iteration method and compared against the Ansys Fluent baseline. To quantitatively validate the solver accuracy, flow data was extracted along a wall-normal line probe spanning the width of the fully developed channel.

As shown in the quantitative line plots in Figure [3.1](#), the velocity and turbulent eddy viscosity profiles generated by the custom FEniCS model match the Ansys reference data impeccably. The custom solver successfully resolves the logarithmic region of the turbulent boundary layer and accurately predicts the turbulent diffusion toward the channel center. This perfect 1:1 alignment provides strict mathematical proof that the core Navier-Stokes solver, the relaxed Eikonal field, and the highly non-linear SA production/destruction terms are functioning correctly before introducing the geometric complexities of the U-bend.

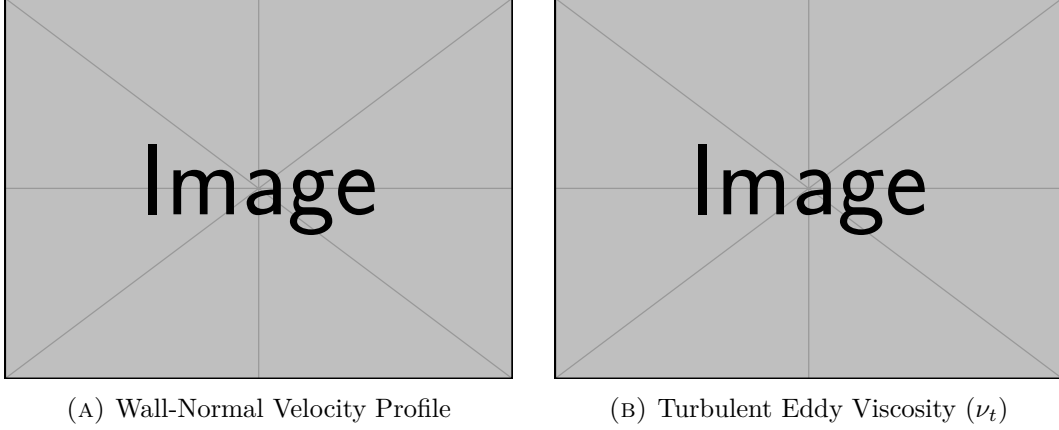


FIGURE 3.1: Quantitative code verification: line plots extracting data across the 2D periodic channel ($Re_H \approx 20,300$). The custom FEniCS SA formulation demonstrates excellent numerical agreement with the commercial Ansys Fluent SA baseline.

3.7 Verification Benchmark 2: U-Bend Geometry

Following the successful channel flow verification, the solver was tested on the highly convective, separating secondary flows of the U-bend geometry evaluated in Chapter 2. It is important to distinguish the physical scope of this specific verification test from the standard Ansys Verification Manual (VMFL048) case.

While the original VMFL048 case is a full 3D simulation, the verification performed here utilized a strictly modified computational domain to ensure efficiency. First, the domain was reduced to a 2D geometry corresponding directly to the symmetry mid-plane of the 3D case. Second, to reduce unnecessary computational overhead during eventual topology optimization, the highly extended straight inlet and outlet sections (1.555 m) of the standard benchmark were explicitly truncated to strictly $30D$ (0.840 m).

To drive the flow, a uniform prescribed velocity ($\mathbf{u}_{in} = 1.42$ m/s) was imposed at the inlet alongside a fully turbulent boundary value ($\tilde{\nu}_{in} = 9.34 \times 10^{-5}$ m²/s), while a zero relative static pressure ($p = 0$ Pa) was assigned to the outlet. While the FEniCS domain is mathematically a 2D planar approximation of the symmetry mid-plane, the 3D physical pipe diameter ($D = 0.028$ m) and geometric radius of curvature ($R_c = 0.125$ m) were strictly retained as the characteristic length scales. Conserving these parameters ensures that the computed Reynolds number ($Re_D \approx 44,700$) and Dean number ($De \approx 15,000$) identically match the 3D Ansys reference case, maintaining non-dimensional consistency and correctly governing the internal secondary flows.

3.7.1 Mesh Resolution Requirements

Because this formulation utilizes a standard Galerkin weak form without spatial upwind stabilization, numerical stability in highly convective flows is strictly dependent on adequate grid resolution. As established in the channel flow benchmark, the SA model requires integration entirely through the viscous sublayer.

Unlike the simple rectangular channel where a programmatic NumPy script was sufficient, the U-bend geometry features curved inner and outer arcs, complex corner transitions, and anisotropic boundary-layer inflation around non-orthogonal walls. To manage this severe geometric complexity while satisfying the $y^+ \approx 1$ resolution requirement, a custom mesh was generated using the advanced open-source mesh generator **Gmsh** [19]. The first element height adjacent to the wall was strictly controlled to 1.20×10^{-5} meters across the entire domain. As documented comprehensively in Appendices A.2.3 and A.2.4, strict independence studies were conducted to explicitly prove that neither the 30D domain truncation nor the spatial grid discretization artificially influenced the secondary flow physics.

3.7.2 FEniCS vs. Ansys Comparison

Having mathematically validated the spatial resolution and boundary placement, an independent Ansys Fluent simulation was executed on the exact shortened 2D domain. This served as a strict 1:1 baseline to verify the accuracy of the custom FEniCS implementation.

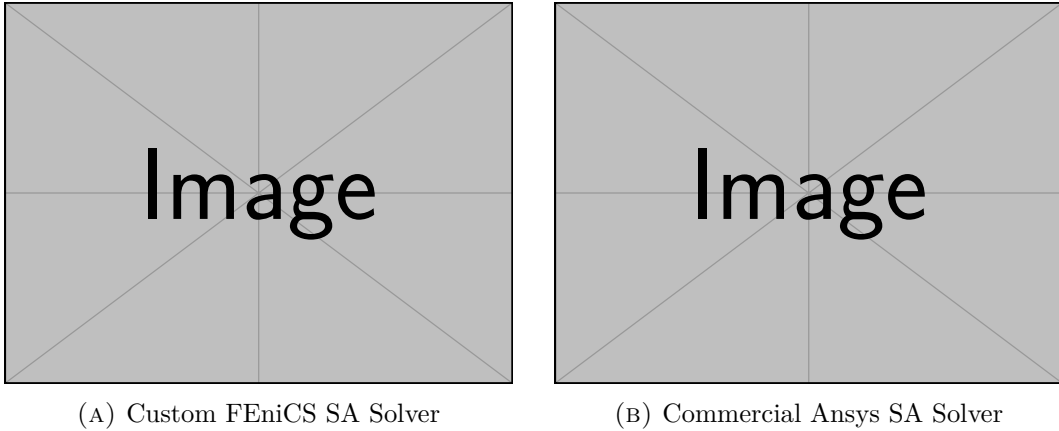


FIGURE 3.2: Code verification: qualitative velocity contours of the truncated 2D U-bend benchmark comparing the custom FEniCS implementation against Ansys Fluent. Both solvers utilize the Spalart-Allmaras model.

While the global flow fields and secondary flow separation bubbles observed in the velocity contours (Figure 3.2) demonstrate excellent visual agreement, Research Question 2 strictly demands *quantitative* validation. To satisfy this requirement, velocity data was extracted along a cross-sectional line probe positioned directly at the bend exit, marking the exact transition from the end of the curvature to the start of the straight outlet section.

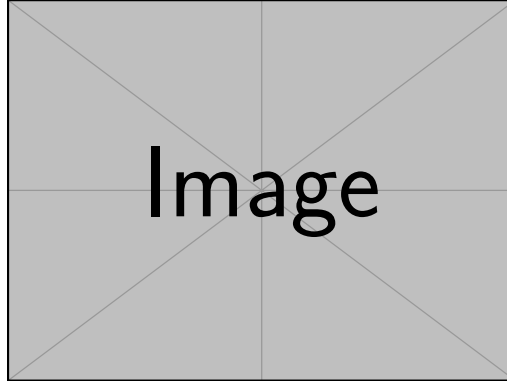


FIGURE 3.3: Quantitative validation: velocity profile across the U-bend exit ($R_c \approx 4.46D$, $Re_D \approx 44,700$, $De \approx 15,000$), comparing the custom FEniCS SA solver directly to the truncated Ansys Fluent baseline.

As shown in Figure 3.3, the velocity profile computed by the custom FEniCS implementation traces the commercial Ansys results closely. The FEniCS implementation accurately replicates the boundary layer growth, the momentum shift toward the outer wall, and the turbulent diffusion predicted by the commercial solver, thereby confirming the mathematical integrity of the code.

3.8 Conclusion

This chapter has detailed the mathematical formulation, stabilization strategy, and rigorous verification of the steady-state Spalart-Allmaras model within the FEniCS framework. By successfully replicating the benchmark results of Ansys Fluent, this implementation directly answers the second research question of this thesis: achieving numerical stability and quantitative agreement in a custom finite element solver, laying a fully consistent foundation for the topology optimization routines developed in the subsequent chapters.

First, numerical stability was successfully achieved without the need for complex mathematical stabilization artifacts. By decoupling the Navier-Stokes and Spalart-Allmaras equations via a rigorous Picard iteration scheme, and utilizing an Incremental Pressure Correction Scheme (IPCS) to manage the convective fluid terms, the highly non-linear system was robustly driven to equilibrium. Supported by high-density mesh resolution and Yoon’s relaxed Eikonal equation, this standard Galerkin formulation completely avoided the numerical oscillations that typically plague turbulent finite-element solvers.

Second, quantitative agreement was thoroughly verified. The solver produced velocity and eddy viscosity profiles that perfectly matched the commercial Ansys Fluent baseline in both simple periodic channel flows ($Re_H \approx 20,300$) and the highly convective, geometrically complex U-bend ($Re_D \approx 44,700$, $De \approx 15,000$). Consequently, this decoupled Picard FEniCS solver is proven to be a mathematically

sound and reliable fluid dynamics engine, specifically structured to support the continuous adjoint topology optimization campaigns presented in Chapters 4 and 5.

Chapter 4

Topology Optimization for Turbulent Flows: Methodology

4.1 Introduction

Building upon the verified Spalart-Allmaras (SA) fluid dynamics solver developed in Chapter 3, this chapter introduces the mathematical framework required to perform Topology Optimization (TO) in the turbulent regime. The core objective of this methodology is to algorithmically determine the optimal distribution of fluid channels within a given design domain to minimize energy dissipation.

This chapter details the Fictitious Domain Method and the Brinkman penalization technique used to conceptualize solid boundaries within a continuous fluid mesh. It explicitly outlines how the standard governing equations for fluid momentum, turbulent transport, and wall distance must be mathematically penalized to accommodate the evolving topology.

To solve this optimization problem in a computationally tractable manner, continuous adjoint sensitivity analysis is required. This chapter presents two distinct adjoint pipelines. First, the "frozen turbulence" (Constant Eddy Viscosity) assumption is established and rigorously justified as the stable baseline optimization engine adopted in this thesis. Subsequently, an advanced, fully coupled "semi-frozen" adjoint formulation is introduced to capture the highly non-linear turbulence production sensitivities. This section details both its physical advantages and its algorithmic limitations.

Following the derivation of these sensitivities, the continuous optimization problem is formally defined, establishing the power dissipation objective function and the strict fluid volume constraints. Finally, the mathematical techniques for density filtering and Heaviside projection, adapted from structural optimization practices, are detailed alongside their FEniCS unified form language (UFL) implementation to ensure the generation of manufacturable designs. All algorithms, penalty formulations, and execution scripts, most notably `TurbulentTO_Frozen.py` and `TurbulentTO_SemiFrozen.py`, are housed within the `FluidTO` directory of the public repository.

4.2 The Fictitious Domain Method and Brinkman Penalization

In standard shape optimization, the solid-fluid boundary is explicitly defined by the mesh, and the nodes are physically moved to improve the design. In density-based topology optimization, however, the computational domain (Ω) remains strictly fixed. To create solid walls and fluid channels within this fixed grid, the Fictitious Domain Method is employed.

Originally applied to Stokes flow by Borrvall and Petersson [12] and extended to the Navier-Stokes equations by Gersborg-Hansen et al. [18], this method assigns a continuous design variable, $\gamma(\mathbf{x}) \in [0, 1]$, to every spatial coordinate in the domain. In this specific implementation, $\gamma = 1$ represents a pure **fluid** channel, while $\gamma = 0$ represents a **solid**, impermeable structure. To force the fluid velocity to zero inside the solid regions without mathematically removing the finite elements from the computation, the fluid is treated as a fictitious porous medium. The solid regions are assigned an artificially high flow resistance, mimicking the physical behavior of a dense matrix.

This flow resistance is quantified by an inverse permeability term, or Brinkman penalty function, denoted as $\alpha(\gamma)$. To ensure the optimization algorithm (which relies on continuous gradients) can transition smoothly between fluid and solid states, $\alpha(\gamma)$ must be a continuous, differentiable interpolation function. This solver utilizes a convex Rational Approximation of Material Properties (RAMP) interpolation scaled between a maximum solid penalty (α_{solid}) and a minimum fluid penalty (α_{fluid}):

$$\alpha(\gamma) = \alpha_{solid} + (\alpha_{fluid} - \alpha_{solid}) \frac{\gamma(1+q)}{\gamma+q} \quad (4.1)$$

where α_{solid} is a sufficiently large penalty factor (e.g., $10^4 \sim 10^5$) that completely stagnates the flow when $\gamma = 0$, and q is a tuning parameter that controls the convexity of the penalization curve [1]. Unlike traditional structural topology optimization which relies on the Solid Isotropic Material with Penalization (SIMP) power-law, fluid optimization generally avoids SIMP because its derivative vanishes at $\gamma = 0$. This zero-gradient condition can prematurely trap the optimizer in local minima. By utilizing the RAMP formulation, a strictly non-zero gradient is guaranteed across the entire design domain, ensuring stable sensitivity analysis. When $\gamma = 1$ (fluid), the interpolation strictly evaluates to α_{fluid} . While α_{fluid} is mathematically non-zero to maintain matrix conditioning, it is several orders of magnitude smaller than α_{solid} , meaning the fluid experiences negligible artificial resistance.

4.3 Penalization of the Governing Equations

To effectively optimize turbulent manifolds, the Brinkman penalty cannot simply be applied to the momentum equations. It must be rigorously integrated into the turbulence model and the wall-distance calculations to ensure the physics remain consistent as the solid boundaries evolve [42].

4.3.1 Modified Reynolds-Averaged Navier-Stokes Equations

The primary application of the Brinkman penalty is the modification of the steady-state, incompressible RANS momentum equation. The inverse permeability $\alpha(\gamma)$ is added as a linear sink term (Darcy friction) to the momentum conservation equation [18]:

$$\rho(\mathbf{u} \cdot \nabla)\mathbf{u} = -\nabla p + \nabla \cdot [(\mu + \mu_t)(\nabla\mathbf{u} + (\nabla\mathbf{u})^T)] - \alpha(\gamma)\mathbf{u} \quad (4.2)$$

It is critical to note the retention of the full symmetric rate-of-strain tensor in this formulation. While the transposed velocity gradient, $(\nabla\mathbf{u})^T$, is frequently simplified out of the governing equations in constant-viscosity laminar solvers due to the divergence-free constraint ($\nabla \cdot \mathbf{u} = 0$), it must be strictly retained here. Because the turbulent eddy viscosity (μ_t) varies drastically across the spatial domain, the effective viscosity is no longer constant, and the transposed term exerts a non-zero physical influence on the momentum balance.

In regions where the optimizer places solid material ($\gamma \rightarrow 0$), the enormous magnitude of $\alpha_{solid}\mathbf{u}$ dominates all inertial and viscous forces, forcing the local velocity $\mathbf{u}$ mathematically to zero. Conversely, in the fluid channels ($\gamma \rightarrow 1$), the penalty term vanishes, recovering the standard RANS equations. The continuity equation ($\nabla \cdot \mathbf{u} = 0$) remains unchanged.

4.3.2 Modified Spalart-Allmaras Transport Equation

As the Fictitious Domain Method forces velocity to zero in the solid regions, the turbulence generation inside those regions should theoretically cease. However, due to numerical diffusion and the presence of the convective terms in the surrounding fluid, turbulent eddy viscosity ($\tilde{\nu}$) can inadvertently bleed into the solid regions. This unphysical behavior destabilizes the solver and heavily distorts the adjoint sensitivity analysis.

To strictly enforce a laminar, zero-turbulence state inside the solid structures, Yoon [42] proposed adding a corresponding penalty sink term to the Spalart-Allmaras transport equation:

$$\mathbf{u} \cdot \nabla \tilde{\nu} = c_{b1}\tilde{S}\tilde{\nu} - c_{w1}f_w \left(\frac{\tilde{\nu}}{y}\right)^2 + \frac{1}{\sigma} [\nabla \cdot ((\nu + \tilde{\nu})\nabla \tilde{\nu}) + c_{b2}(\nabla \tilde{\nu})^2] - \alpha_{\tilde{\nu}}(\gamma)\tilde{\nu} \quad (4.3)$$

Here, $\alpha_{\tilde{\nu}}(\gamma)$ follows a similar RAMP interpolation profile. As $\gamma \rightarrow 0$, this massive sink term overwhelmingly destroys any turbulent working variable $\tilde{\nu}$ attempting to cross into the solid domain, effectively isolating the fluid turbulence to the active flow channels.

It must be noted that this formulation intentionally diverges from the exact mathematical notation presented by Yoon. In the original literature, the turbulence penalty was defined using a traditional SIMP power-law interpolation, formatted exactly as $\alpha_{\tilde{\nu}_t}\gamma^{n_{\tilde{\nu}_t}}\tilde{\nu}$, where $n_{\tilde{\nu}_t}$ acts as the penalization exponent and $\gamma = 1$ designates the solid domain (the opposite convention to the one adopted herein). However, this exponent is strictly omitted in Equation 4.3 to maintain mathematical consistency with the custom framework developed for this thesis. Because the

penalty multiplier $\alpha_{\tilde{\nu}}(\gamma)$ is already evaluated using the strictly bounded RAMP function (Equation 4.1), applying an additional polynomial exponent would result in a mathematically redundant double penalization. This would artificially distort the carefully tuned convexity of the RAMP curve and reintroduce the zero-gradient issues that the RAMP formulation was chosen to eliminate.

4.3.3 Modified Relaxed Eikonal Equation

The Spalart-Allmaras model is highly sensitive to the wall distance, y . While differential equations for wall distance computation are well-established in standard CFD [38], topology optimization introduces a unique challenge: the location of the wall is not a static mesh boundary, but rather the dynamically evolving $\gamma = 0$ contour. Therefore, the relaxed Eikonal equation must also be penalized to recognize the fictitious solid as a physical boundary.

To ensure that the wall distance correctly evaluates to $y = 0$ inside the solid regions, the intermediate field variable G must be forced to its upper boundary limit, G_0 . This is achieved by adding a third penalty term to the Eikonal formulation:

$$\nabla G \cdot \nabla G + \sigma_w G (\nabla \cdot \nabla G) = (1 + 2\sigma_w)G^4 + \alpha_G(\gamma)(G - G_0) \quad (4.4)$$

In the solid region, the high value of $\alpha_G(\gamma)$ tightly binds the local solution to $G = G_0$. Recalling the algebraic recovery function $y = 1/G - 1/G_0$, this correctly evaluates to an absolute wall distance of $y = 0$ anywhere the optimizer has placed structural material [42].

As with the Spalart-Allmaras model, this boundary sink term structurally diverges from Yoon’s original SIMP-based notation. The reference literature explicitly defines this term as $\alpha_G \gamma^{n_G} (G - G_0)$, utilizing both the penalization exponent n_G and the reversed solid convention. However, this exponent is strictly omitted in Equation 4.4. Consistent with the rationale established in the previous section, applying an additional polynomial over the already bounded RAMP multiplier $\alpha_G(\gamma)$ would result in a mathematically redundant double penalization, artificially distorting the desired convexity of the formulation.

4.4 Sensitivity Analysis and the Frozen Turbulence Assumption

With the forward state equations fully defined and penalized, the topology optimization algorithm requires the gradient (sensitivity) of the objective function with respect to the design variable γ . Because evaluating finite differences for every mesh element is computationally impossible, the continuous adjoint method is employed. The adjoint method solves a supplementary set of linear equations to efficiently compute the exact gradient of the objective function [15].

However, calculating the fully coupled, exact adjoint for turbulent flows is mathematically formidable. As demonstrated by Zymaris et al. [45], formulating the exact continuous adjoint for the Spalart-Allmaras model requires differentiating the

highly non-linear production and destruction terms, accounting for the variations in the distance function, and managing the intense cross-coupling with the momentum equations. The complexity of these exact adjoints frequently introduces severe numerical stiffness and divergence during the iterative optimization process.

To make the gradient calculation both numerically robust and computationally feasible, the primary optimization workflows in this implementation rely on the **Frozen Turbulence Assumption**, frequently referred to in the literature as the Constant Eddy Viscosity (CEV) assumption [33, 41].

4.4.1 Mechanism of Frozen Turbulence

The frozen turbulence assumption simplifies the sensitivity analysis by decoupling the turbulence transport from the design update. During the adjoint formulation, the turbulent eddy viscosity field (ν_t) generated by the forward SA solver is assumed to be locally independent of infinitesimally small changes in the design variable (γ). Mathematically, this implies that during the calculation of the adjoint sensitivities:

$$\frac{\partial \nu_t}{\partial \gamma} \approx 0 \quad (4.5)$$

By treating the eddy viscosity field as frozen (a spatially varying but constant parameter), the complex adjoints of the Spalart-Allmaras and Eikonal equations do not need to be solved. The optimizer only solves the standard fluid adjoint equations (utilizing the effective viscosity $\mu + \mu_t$) to determine how a change in γ affects the pressure drop [42].

4.4.2 Justification and Limitations

The primary advantage of the frozen turbulence approach is not that it is universally “better”, but that it makes the optimization problem solvable within a reasonable numerical and implementation budget. Relative to a fully coupled turbulent adjoint, it offers far greater numerical stability, a far smaller implementation burden, and a materially lower computational cost per design iteration.

However, it is crucial to recognize its physical limitations. Because the algorithm ignores how a change in the wall shape will affect future turbulence generation, the gradients provided to the optimizer are approximations rather than exact derivatives. Recent studies have demonstrated that while the frozen assumption performs well in attached flows and moderate pressure gradients, it can alter the accuracy of the sensitivities in regimes characterized by massive flow separation or highly anisotropic shear layers, potentially causing the optimizer to settle into sub-optimal local minima [41]. In that sense, the frozen adjoint is defended here on tractability grounds rather than on a claim of physical completeness.

4.5 The Coupled "Semi-Frozen" Adjoint Formulation

In classical turbulent topology optimization, the widely used "frozen turbulence" assumption neglects the variation of turbulent viscosity with respect to the design field to maintain numerical stability [15]. However, literature indicates that overcoming this assumption by differentiating the Spalart-Allmaras equations can yield more physically accurate sensitivity derivatives, particularly in flows dominated by severe separation [45, 29].

To address the physical limitations of the frozen assumption, an advanced "semi-frozen" continuous adjoint formulation was additionally developed for this thesis. Unlike the baseline approach, this formulation explicitly re-couples the Spalart-Allmaras turbulence transport equation into the adjoint sensitivity analysis, allowing the optimizer to mathematically anticipate how topological changes will generate or suppress turbulent kinetic energy.

4.5.1 The Monolithic State and Geometric Decoupling

Deriving the coupled adjoint requires defining a monolithic forward state vector. While the frozen adjoint only considers the velocity and pressure fields $\mathbf{s} = [\mathbf{u}, p]^T$, the coupled adjoint expands the state vector to include the turbulent working variable: $\mathbf{s} = [\mathbf{u}, p, \tilde{\nu}]^T$. During the adjoint solve, a corresponding monolithic adjoint vector $\mathbf{s}^* = [\mathbf{u}^*, p^*, \tilde{\nu}^*]^T$ is computed simultaneously.

It is critical to note that this formulation seeks a computational middle ground and is termed "semi-frozen" because the geometric wall-distance variable (G from the relaxed Eikonal equation) is intentionally omitted from the monolithic state. While the ultimate mathematical formulation of an exact continuous adjoint requires a fully coupled four-field vector $\mathbf{s} = [\mathbf{u}, p, \tilde{\nu}, G]^T$, directly differentiating the distance field is notoriously difficult and computationally expensive [27]. The relaxed Eikonal equation is fundamentally geometric and lacks the convective time-scales of the fluid equations. Attempting to differentiate its highly non-linear G^4 source term alongside the Navier-Stokes system frequently results in severe matrix ill-conditioning. Therefore, explicitly decoupling G serves as an intentional stabilization strategy, preserving the condition number of the monolithic system while successfully recovering the dominant convective turbulence sensitivities.

4.5.2 Physical Advantages

By including $\tilde{\nu}$ in the adjoint state, the optimizer is no longer blind to turbulence generation. The adjoint equations actively differentiate the production term ($c_{b1}\tilde{S}\tilde{\nu}$) with respect to the velocity field. Because turbulence production in the Spalart-Allmaras model is directly driven by fluid shear (vorticity) [34], the optimizer mathematically detects that placing a sharp solid boundary in a high-velocity stream will generate massive shear, drastically increasing turbulent dissipation. Consequently, the semi-frozen adjoint actively smooths out geometric profiles to mitigate flow sep-

aration *before* it occurs, rather than passively reacting to a pre-calculated, frozen flow field.

4.5.3 Mathematical Smoothing and FEniCS UFL Limitations

Implementing this monolithic adjoint in FEniCS introduces significant mathematical challenges. FEniCS utilizes the Unified Form Language (UFL) to perform automated symbolic differentiation ($\partial J/\partial \mathbf{s}$). However, this requires the weak forms to be strictly continuously differentiable (C^1 continuous).

As widely documented in continuous adjoint literature [45, 29], the standard Spalart-Allmaras model contains theoretically non-differentiable functions. Specifically, the model relies on absolute values to calculate the vorticity magnitude scalar, and uses strict ‘min()’/‘max()’ bounding functions to prevent numerical singularities in the wall destruction terms. If passed directly into the UFL symbolic derivative engine, the Jacobian assembly fails.

To overcome this, the custom FEniCS implementation utilizes rigorous hyperbolic smoothing approximations. The absolute value function $|x|$ is mathematically replaced by a smoothed hyperbola $\sqrt{x^2 + \epsilon^2}$, where ϵ is a vanishingly small constant. Similarly, hard ‘min’/‘max’ bounds are substituted with smooth, differentiable hyperbolic tangent (tanh) transitions. These mathematical approximations ensure the UFL engine can cleanly invert the monolithic Jacobian matrix without triggering step-function singularities.

4.5.4 Theoretical Vulnerability to Boundary Singularities

While the semi-frozen adjoint successfully captures complex interior fluid mechanics, exact adjoint formulations for turbulence models are notoriously susceptible to numerical noise at domain boundaries [15, 29].

Because the coupled adjoint is hypersensitive to velocity gradients, it can become mathematically fixated on artificial shear generated at physical inlets. When boundary conditions abruptly transition from a prescribed inlet velocity to a stationary solid wall, the resulting step discontinuity in vorticity acts as an artificial, localized source of turbulence production. If left unconstrained, the optimizer may attempt to minimize the objective function by heavily damping these specific boundary nodes rather than resolving the interior routing. The practical manifestation of this mathematical vulnerability, and its subsequent requirement for strict non-design buffer domains ($\bar{\gamma} = 1$), will be evaluated during the benchmark studies in Chapter 5.

4.6 Optimization Problem Formulation

Regardless of whether the continuous sensitivities are derived using the baseline frozen assumption or the advanced semi-frozen adjoint, the foundational goal of the optimization remains identical. The topology optimization task is formulated as a mathematical programming problem, where the optimizer must find a spatial

distribution of the design variable $\gamma(\mathbf{x})$ that minimizes a specific performance metric subject to physical and geometric constraints.

4.6.1 The Objective Function: Power Dissipation

For fluid manifolds, the primary engineering goal is to guide the fluid from the inlets to the outlets with the minimum possible pressure drop. In incompressible flow, minimizing the total pressure drop across the boundaries is mathematically equivalent to minimizing the total power dissipated by the fluid within the domain [12, 1].

The objective function, J , is formulated by integrating the internal viscous energy dissipation and the artificial Brinkman friction over the entire computational domain Ω :

$$J(\mathbf{u}, \gamma) = \int_{\Omega} \left[\frac{1}{2}(\mu + \mu_t) (\nabla \mathbf{u} + (\nabla \mathbf{u})^T) : (\nabla \mathbf{u} + (\nabla \mathbf{u})^T) + \alpha(\gamma) \mathbf{u} \cdot \mathbf{u} \right] d\Omega \quad (4.6)$$

The first term in the integral represents the true physical dissipation caused by the effective fluid viscosity (both laminar shear and turbulent eddy interactions). The second term represents the artificial energy lost when the fluid attempts to penetrate the fictitious solid regions defined by $\alpha(\gamma)$. As the optimizer seeks to minimize J , it is naturally driven to carve out clear fluid channels ($\gamma \rightarrow 1$, where $\alpha \rightarrow \alpha_{fluid}$) and avoid pushing flow through "gray" or solid regions ($\gamma \rightarrow 0$, where $\alpha \rightarrow \alpha_{solid}$), thereby generating a highly efficient topology.

4.6.2 Volume Constraint

If the optimizer were allowed to minimize J without restrictions, the trivial and optimal solution would be to set $\gamma = 1$ everywhere, effectively removing all solid material and turning the entire domain into one massive fluid channel. Because the goal is to design a targeted manifold or pipe system, a strict volume constraint must be imposed to limit the amount of fluid allowed in the design space [12].

Recalling that $\gamma = 1$ represents pure fluid and $\gamma = 0$ represents solid, the total volume of the fluid, V_f , can be calculated by simply integrating the design variable over the domain [1]:

$$V_f(\gamma) = \int_{\Omega} \gamma d\Omega \quad (4.7)$$

To ensure the final design does not exceed a specified maximum fluid capacity, the volume fraction constraint is enforced as:

$$g(\gamma) = \frac{\int_{\Omega} \gamma d\Omega}{V_{domain}} - V_{max} \leq 0 \quad (4.8)$$

where V_{domain} is the total volume of the computational space, and V_{max} is the maximum allowable fluid volume fraction (e.g., $V_{max} = 0.3$ strictly limits the fluid channels to occupying only 30% of the design space).

4.6.3 The Complete Continuous Optimization Problem

Combining the objective function, the governing equations, and the constraints, the continuous topology optimization problem for turbulent flow is formally stated as:

$$\begin{aligned}
 & \underset{\gamma(\mathbf{x}) \in [0,1]}{\text{minimize}} && J(\mathbf{u}, \gamma) \\
 & \text{subject to} && \text{Modified RANS Equations (Eq. 4.2)} \\
 & && \text{Modified Spalart-Allmaras Equation (Eq. 4.3)} \\
 & && \text{Modified Relaxed Eikonal Equation (Eq. 4.4)} \\
 & && g(\gamma) \leq 0
 \end{aligned} \tag{4.9}$$

To solve this constrained minimization problem, the FEniCS implementation utilizes the Method of Moving Asymptotes (MMA) [36], an optimization algorithm specifically designed for handling large-scale, gradient-based topology optimization problems. As a gradient-driven optimizer, MMA approximates the highly non-linear design space using convex asymptotes. To maintain solver stability and prevent chaotic oscillations between design iterations, the algorithm relies on strict “move limits”, which cap the maximum allowable change in the design variable (γ) during any single optimization step.

4.7 Density Filtering and Projection

A well-known mathematical complication in density-based topology optimization is the lack of existence of a classical solution. If the design space is left unconstrained, the optimizer will tend to create infinitely thin micro-structures (often appearing as numerical “checkerboarding” in finite element meshes) to maximize performance. To guarantee a mesh-independent and manufacturable design, a minimum length scale must be enforced.

4.7.1 The Discontinuous Galerkin Helmholtz Filter

To prevent checkerboarding, a density filter is applied to the raw design variable generated by the optimizer, γ_{raw} . In standard topology optimization, the design variable is typically defined at the mesh nodes using continuous linear basis functions (Continuous Galerkin, CG1). This allows the filtering process to be governed by the standard continuous Helmholtz partial differential equation [24]:

$$-r^2 \nabla^2 \tilde{\gamma} + \tilde{\gamma} = \gamma_{raw} \tag{4.10}$$

where $\tilde{\gamma}$ is the filtered design variable and r represents the internal Helmholtz PDE coefficient. In classical topology optimization literature [24], it is well-established that to make the PDE filter approximate a standard spatial convolution filter with a nominal physical radius R , this coefficient must be mathematically scaled. Therefore, the internal coefficient is defined as $r = \frac{R}{2\sqrt{3}}$, where R is the explicit physical length scale enforced upon the design.

However, defining the density continuously across elements poses a severe physical problem for fluid topology optimization. A continuous nodal definition causes the design variable (and consequently, the Brinkman penalty) to interpolate smoothly across the interior of boundary elements. This creates a semi-permeable "gray" gradient inside the element, which allows fluid velocity to numerically leak through the structural walls, violating mass conservation. To strictly prevent this mass leakage, this implementation discretizes the density field γ using piecewise constant elements (Discontinuous Galerkin, DG0). By assigning a single, uniform density value to the entire element, the structural boundaries remain perfectly sharp at the element edges.

Consequently, the standard continuous Helmholtz filter (Equation 4.10) can no longer be applied, as it requires continuously differentiable basis functions. The filter must be mathematically adapted to account for the discontinuous boundaries between the DG0 elements. This is achieved using an interior penalty method across the element facets (internal boundaries, Γ_{int}) [9].

Unlike the physical governing equations presented previously, the DG0 filter is a strictly numerical finite element construct and must be defined explicitly in its weak integral formulation. Because the density is constant within each element, the continuous spatial gradient ($\nabla \tilde{\gamma}$) strictly vanishes, meaning standard volume diffusion is mathematically replaced by interior facet jump penalties [20]. As implemented in the custom FEniCS solver, an artificial diffusion term scaled by α_{dg} is applied exclusively to these jumps:

$$\int_{\Gamma_{int}} r^2 \frac{\alpha_{dg}}{h_{avg}} \llbracket \tilde{\gamma} \rrbracket \cdot \llbracket v \rrbracket dS + \int_{\Omega} \tilde{\gamma} v dx = \int_{\Omega} \gamma_{raw} v dx \quad (4.11)$$

where v is the test function, $\llbracket \cdot \rrbracket$ denotes the jump operator across a facet, and h_{avg} is the average cell size. This formulation, alongside the Lazarov radius scaling factor, is mapped directly into FEniCS Python syntax as demonstrated in Listing 4.1.

LISTING 4.1: Implementation of the DG0 Helmholtz filter and interior penalty scaling. Python dictionary lookups have been simplified for readability. Extracted from `TurbulentTO_Frozen.py`.

```
# r_filter corresponds to the nominal physical radius (R).
# Dynamically calculated to span across multiple mesh cells.
r_filter = compute_filter_base_length_from_config() *
    FILTER_RADIUS_IN_CELLS

# r is the internal PDE coefficient.
# Scaled by 1/(2*sqrt(3)) per the Lazarov et al. (2011) proof.
r = r_filter / (2.0 * 3.0**0.5)

u_filter = TrialFunction(DensitySpace)
v_filter = TestFunction(DensitySpace)
filter_in = Function(DensitySpace)

# h_avg represents the average cell diameter at the facet.
# Dynamically scales the interior facet jump penalty.
h = CellDiameter(mesh)
```

```

h_avg = (h("+") + h("-")) / 2.0

def pde_filter(input_field, output_field):
    """DGO Helmholtz filter implementation using interior facet
    jumps."""
    alpha_dg = 4.0
    helmholtz = (
        r**2 * (alpha_dg / h_avg * dot(jump(v_filter, n), jump(
            u_filter, n))) * dS
        + u_filter * v_filter * dx
        - filter_in * v_filter * dx
    )
    assign(filter_in, input_field)
    solve(lhs(helmholtz) == rhs(helmholtz), output_field)
    return output_field

```

4.7.2 Heaviside Projection

While the Helmholtz filter successfully imposes a minimum length scale, it inherently smooths the design field, resulting in a large "gray" transition zone between the solid ($\gamma = 0$) and fluid ($\gamma = 1$) regions. To recover a crisp, discrete topology, a continuous Heaviside projection is applied to the filtered variable [6].

This implementation utilizes a hyperbolic tangent ($\tanh$) projection function [1]:

$$\bar{\gamma} = \frac{\tanh(\beta\eta) + \tanh(\beta(\bar{\gamma} - \eta))}{\tanh(\beta\eta) + \tanh(\beta(1 - \eta))} \quad (4.12)$$

where $\bar{\gamma}$ is the projected design variable, $\eta \in [0, 1]$ is the projection threshold (typically 0.5), and β is a steepness parameter. As $\beta \rightarrow \infty$, the function approaches a strict Heaviside step function. The FEniCS implementation of this projection is provided in Listing 4.2.

LISTING 4.2: Implementation of the continuous hyperbolic tangent projection. Extracted from `TurbulentTO_Frozen.py`.

```

def projection(rho_design, eta_proj):
    """Hyperbolic tangent (tanh) projection used to sharpen the
    filtered design."""
    return (
        tanh(BETA_PROJ * Constant(eta_proj))
        + tanh(BETA_PROJ * (rho_design - Constant(eta_proj)))
    ) / (
        tanh(BETA_PROJ * Constant(eta_proj))
        + tanh(BETA_PROJ * (Constant(1.0) - Constant(eta_proj)))
    )

```

4.8 Algorithmic Workflow and Solver Implementation

While the previous sections define the continuous mathematical problem, executing the optimization requires a highly robust discrete, iterative algorithm. The

computational framework developed for this thesis originated from a basic laminar topology optimization template provided by KU Leuven. However, to support the severe non-linearities of the turbulent regime, this architecture was fundamentally restructured. The resulting custom optimization engine natively manages the segregated Spalart-Allmaras turbulence model, explicit physics decoupling via Picard iterations, dual continuous adjoint pipelines, and the discontinuous DG0 interior penalty filter.

4.8.1 Decoupling the Physics: The Picard Iteration Scheme

As established in Chapter 3, to guarantee solver stability under the severe non-linearities of the turbulence model and Brinkman penalization, the forward governing equations are rigorously decoupled. The pressure and velocity fields are managed via the IPCS algorithm, while the fluid and turbulence transport equations are decoupled via a Picard iteration scheme.

During each Picard step within the optimization loop, the IPCS solver advances the flow field using the eddy viscosity from the previous iteration. The updated velocity field is then passed to the SA solver to calculate a new eddy viscosity field. Only after this alternating forward loop has completed its designated sweeps is the flow field considered equilibrated, at which point the continuous adjoint routine is triggered to compute the topological sensitivities.

4.8.2 Forward State Convergence and Adjoint Accuracy

A critical requirement of the frozen turbulence assumption is its reliance on a fully converged forward physical state. Because the frozen adjoint calculates gradient sensitivities based on a static snapshot of the eddy viscosity field (ν_t), any incomplete convergence in the forward flow solver directly corrupts the gradient assembly. During the optimization loop, a single Picard sweep between the IPCS flow solver and the Spalart-Allmaras transport equation is generally insufficient to reach local equilibrium. To prevent unconverged physical states from misleading the optimizer, a strict minimum of three Picard iterations is enforced per design cycle. This ensures that the frozen adjoint sensitivities are derived from an equilibrated velocity and turbulence field before the Method of Moving Asymptotes (MMA) updates the topology.

4.8.3 Python Implementation and Continuation Strategy

To ensure convergence to a physically meaningful binary design, the algorithm iterates over a strict continuation schedule. As the optimization progresses, the Brinkman penalty parameter (q) and the projection steepness (β) are iteratively increased, while the MMA move limits are proportionally tightened.

While the exact gradient assembly differs between the two frameworks, the overarching algorithmic architecture remains consistent. The global optimization routine, illustrated below via the baseline `TurbulentT0_Frozen.py` execution script, is demonstrated in Listing 4.3:

LISTING 4.3: Algorithmic architecture of the iterative topology optimization loop using the frozen turbulence assumption. Code structure has been simplified to highlight the core physics and adjoint pipelines. Extracted from `TurbulentTO_Frozen.py`.

```
for stage_idx, q_val in enumerate(Q_PENAL_SCHEDULE):
    # Update continuation parameters for the current stage
    BETA_PROJ.assign(BETA_PROJ_SCHEDULE[stage_idx])
    q_penal.assign(q_val)

    while inner_count < max_iters_now and not objective_converged:
        # 1. Spatial Filtering & Penalized Wall-Distance Update
        rho_f = pde_filter(rho, rho_f)
        update_wall_distance_field()

        # 2. Forward Physics (Outer Picard Loop for Decoupling)
        for picard_idx in range(PICARD_STEPS):
            solve_forward_with_recovery(...) # IPCS Navier-Stokes solve

            sa_model.construct_forms(w_fwd.sub(0))
            sa_model.solve_turbulence_model()
            sa_model.update_variables(relaxation=NUT_RELAXATION_FACTOR)

            # Lock in the updated eddy viscosity for the frozen assumption
            nu_tilde_frozen.assign(sa_model.nu_tilde0)

            # Final flow solve locking in the converged eddy viscosity
            solve_forward_with_recovery(...)

        # 3. Sensitivity Analysis (Continuous Adjoint)
        solve_adjoint(objective_adjoint_form)

        # 4. Gradient Assembly & DGO Filtering
        unfiltered_gradient.vector()[:] = assemble(objective_ddx[:])
        filtered_gradient = pde_filter(unfiltered_gradient,
                                       filtered_gradient)

        # 5. Method of Moving Asymptotes (MMA) Design Update
        (xmma, ...) = mmasub(mmma, num_mma, iter_count, xval, xmin,
                             xmax, ...)
        rho.vector()[:] = xmma[:, 0]

    inner_count += 1
```

Chapter 5

Topology Optimization for Turbulent Flows: Results and Cross-Code CFD Verification

5.1 Introduction

With the mathematical framework for turbulent topology optimization established in Chapter 4, this chapter presents the resulting optimized topologies and rigorously evaluates their fluid dynamic performance. The objective is not merely to demonstrate that turbulent optimization yields different geometries than laminar assumptions, but to verify the physical credibility of the Spalart-Allmaras (SA) driven designs through independent computational fluid dynamics (CFD) analysis.

To achieve this, a three-stage evaluation pipeline is utilized. First, the turbulent topology optimization is executed within the custom FEniCS framework, analyzing the objective function convergence, fluid routing, and eddy viscosity distributions. Second, the optimized continuous design fields are extracted as sharp, discrete geometries and exported to Ansys Fluent. These geometries are re-analyzed using the identical SA model to quantify cross-code consistency and assess the impact of geometry extraction. Finally, the designs are evaluated in Ansys using the higher-fidelity SST $k - \omega$ (and standard $k - \epsilon$) models to determine the sensitivity of the performance metrics to the underlying turbulence closure, particularly in regimes characterized by high Dean numbers and flow separation.

5.2 Boundary Conditions and Evaluation Metrics

To accurately capture the physics of internal turbulent manifolds, the boundary conditions for all benchmark cases strictly utilize uniform inlet velocity profiles combined with a prescribed turbulence intensity (TI), diverging from the fully developed parabolic profiles typical of laminar topology optimization. Furthermore, to prevent the optimizer from exploiting artificial shear layers generated directly at the boundary constraints, non-design fluid regions ($\gamma = 1$) are enforced at all inlets

and outlets. This allows the turbulent flow to naturally develop before interacting with the actively optimized design domain.

The primary performance metric is the total power dissipation (or equivalently, the total pressure drop for incompressible, fixed-flow-rate systems). Direct comparison of the raw objective function J between FEniCS and Ansys requires careful interpretation. The FEniCS objective integrates both physical viscous dissipation and the artificial Brinkman penalty across a diffuse field. Conversely, Ansys evaluates purely physical dissipation on a body-fitted mesh bounded by sharp walls. Therefore, while the FEniCS objective history demonstrates the internal convergence of the optimizer, the ultimate validation metric relies on the post-processed hydraulic power loss calculated on the extracted Ansys geometries.

Geometry extraction inherently introduces uncertainties. The continuous design variable $\bar{\gamma}$ is thresholded at a specific iso-value to define the solid-fluid interface. Any discrepancies in dissipation between FEniCS and Ansys stem from a combination of discretization differences, solver architecture, and this thresholding process itself.

5.3 Benchmark 1: The Diffuser

The diffuser serves as the foundational benchmark to evaluate separation control in a purely expanding flow, isolating shear layer dynamics from curvature-induced secondary flows.

5.3.1 FEniCS-SA Optimization Results

The optimization relies on the frozen turbulence adjoint to suppress recirculation zones and minimize the pressure drop across the expansion. [Insert text analyzing the objective function plot: note how rapidly the objective drops initially before asymptoting].

Figure [X] illustrates the final projected topology, alongside the velocity magnitude and eddy viscosity fields. [Insert critical analysis of the SA field: Does the optimizer successfully maintain attached flow? Does the eddy viscosity field indicate an over-prediction of diffusion that artificially stabilizes the flow?]

5.3.2 Cross-Code Verification and Model Sensitivity

The thresholded geometry was imported into Ansys Fluent. Table [X] details the power dissipation evaluated across the different solvers and turbulence models.

Comparing FEniCS-SA to Ansys-SA reveals a relative difference of [X]%. This discrepancy highlights the impact of moving from a Brinkman-penalized diffuse interface to a wall-resolved, body-fitted boundary. When analyzed with the SST $k-\omega$ model, the total dissipation shifts by [X]%. [Insert critical analysis: Does the SST model predict flow separation that the SA model missed during optimization? If the SA design fails to perform well under SST conditions, acknowledge this openly as a limitation of the one-equation closure in expanding adverse pressure gradients].

5.4 Benchmark 2: The Double Pipe

The double pipe configuration introduces complex merging and splitting dynamics, assessing the optimizer’s ability to mitigate internal junction losses and mixing dissipation without the complications of global curvature.

5.4.1 FEniCS-SA Optimization Results

[Insert analysis of FEniCS results. Focus on the central junction. How does the optimizer route the flow to avoid stagnation zones? Reference the ν_t field to show where turbulence production is localized.]

5.4.2 Cross-Code Verification and Model Sensitivity

Table [X] presents the comparative CFD metrics for the double pipe geometry. [Insert analysis of the Ansys results. Evaluate if the sharp geometry extraction preserved the smooth junction routing. Compare SA vs SST $k-\omega$: Are the mixing losses at the intersection heavily dependent on the chosen turbulence model?]

5.5 Benchmark 3: The Pipe Bend

To strictly evaluate the framework’s performance in curvature-dominated regimes, the pipe bend benchmark proposed by Alexandersen et al. (2026) is utilized. This case incorporates explicit non-design regions and forces the fluid through a severe geometric deflection, triggering strong streamline curvature.

5.5.1 Dean Number and Theoretical Limits

As established in Chapter 2, the standard Boussinesq approximation utilized by the SA model struggles to accurately capture the anisotropic stress tensors generated by strong secondary flows. The operational Dean number (De) for this benchmark is calculated as [Insert Value], based on the characteristic inlet diameter and the mean radius of curvature generated by the optimizer.

5.5.2 Optimization and Re-analysis

[Insert analysis of the FEniCS TO results].

Re-analysis in Ansys provides critical insight into the physical limits of the SA-optimized design. As shown in Table [X], the performance gap between Ansys-SA and Ansys SST $k-\omega$ is [Insert Result]. [If the gap is large: This discrepancy confirms the theoretical limitations outlined in Chapter 2. The SA-driven optimizer, blind to curvature-induced anisotropy, generates a topology that underperforms when subjected to higher-fidelity turbulence models. This is not a failure of the numerical implementation, but a demonstrated physical limit of utilizing a one-equation linear eddy viscosity model for highly curved manifolds.]

5.6 Benchmark 4: The U-Bend (Advanced Separation)

The U-bend benchmark forces a 180-degree flow reversal, combining intense curvature with inevitable flow separation at the inner radius. [Present the FEniCS results and Ansys comparisons. Focus on whether the optimizer managed to delay separation, and whether SST $k - \omega$ validates or rejects the SA-generated inner wall contour].

5.7 Evaluation of the Coupled “Semi-Frozen” Adjoint

While the frozen adjoint serves as the primary optimization engine due to its numerical tractability, an advanced “semi-frozen” adjoint was formulated to explicitly capture turbulence production sensitivities.

However, evaluating this coupled formulation reveals significant practical limitations. As observed in Figure [X], while the coupled adjoint mathematically detects shear-driven turbulence, exact continuous adjoints for turbulence models are notoriously susceptible to numerical noise at domain boundaries. Even with non-design regions implemented, the coupled sensitivities frequently become dominated by artificial shear concentrations at fluid-solid transitions. Consequently, the optimizer tends to prioritize mitigating these localized mathematical artifacts rather than optimizing the global fluid routing. Therefore, despite its theoretical superiority, the frozen adjoint remains the more robust and reliable approach for the manifolds studied in this thesis.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

The primary objective of this thesis was to develop and evaluate a turbulent topology optimization framework utilizing the Spalart-Allmaras (SA) turbulence model within FEniCS, specifically investigating its physical credibility and limits in internal flow routing.

The investigation into the theoretical limits of the SA model confirmed its utility and its vulnerabilities. While its wall-resolved nature and relative numerical stability make it highly attractive for adjoint-based optimization frameworks, its reliance on the linear Boussinesq approximation fundamentally limits its accuracy in curvature-dominated flows. The model adequately resolves attached flows and moderate expansions, but loses fidelity in regimes characterized by high Dean numbers and strong anisotropic shear.

Implementation of this framework in FEniCS proved successful. By employing a rigorously decoupled Picard iteration scheme, an IPCS-based Navier-Stokes solver, and a stabilized relaxed Eikonal equation for wall-distance computation, the highly non-linear SA model was successfully integrated as the forward state engine for topology optimization. The utilization of a strictly discontinuous (DG0) density filter and Heaviside projection successfully enforced minimum length scales and prevented unphysical mass leakage across structural boundaries.

The most critical evaluation of the framework was established through independent cross-code CFD verification. Rather than merely comparing turbulent topologies to laminar equivalents, the extracted SA-optimized designs were re-analyzed in Ansys Fluent. The cross-code comparison between FEniCS-SA and Ansys-SA validated the numerical integrity of the custom solver, with discrepancies primarily attributed to the transition from a Brinkman-penalized diffuse field to a sharp, body-fitted geometry. However, re-analysis utilizing the SST $k - \omega$ model quantified the sensitivity of the designs to the optimization turbulence model. In benchmarks with moderate streamline curvature (e.g., the diffuser and double pipe), the SA-derived topologies performed robustly under higher-fidelity evaluation. Conversely, in the highly curved pipe bend benchmark, the limitations of the SA optimization became

evident, verifying that topologies optimized under one-equation assumptions may be sub-optimal when evaluated against the complex secondary flows captured by two-equation models.

6.2 Limitations

An honest evaluation of this methodology reveals several important limitations:

1. **The Frozen Adjoint Approximation:** The optimization relies on the frozen turbulence assumption. While justified by numerical tractability and solver stability, it is an approximation. By ignoring the derivative of the eddy viscosity with respect to the design field, the optimizer cannot mathematically anticipate future turbulence production. In flows dominated by massive separation, this can lead the optimizer to sub-optimal local minima.
2. **Geometry Extraction Uncertainties:** The transition from a continuous density field ($\gamma \in [0, 1]$) to a Boolean CAD geometry introduces ambiguity. The chosen thresholding value dictates the final fluid volume and wall placement. Consequently, performance differences between FEniCS and Ansys are inextricably linked to the manual geometry extraction process, making it difficult to isolate pure solver discrepancies.
3. **Lack of Experimental Validation:** While cross-code verification against commercial CFD strongly corroborates the numerical implementation, it is not equivalent to physical validation. The true performance of these turbulent manifolds requires future validation against experimental pressure-drop and flow-field measurements.

6.3 Future Work

The framework developed in this thesis establishes a stable foundation for several avenues of future research:

- **Three-Dimensional Topologies:** The current 2D implementation should be extended to 3D. Turbulent secondary flows (e.g., Dean vortices) are inherently 3D phenomena; optimizing in 3D is strictly necessary to fully evaluate the geometric limits of the turbulence models.
- **Conjugate Heat Transfer (CHT):** With the turbulent fluid mechanics framework verified, the natural next step is to incorporate the energy equation to optimize active cooling channels and heat sinks.
- **Advanced Adjoint Stabilization:** Further research is required to stabilize the monolithic “semi-frozen” adjoint. If the mathematical artifacts at the boundaries can be mitigated, explicitly coupling the turbulence production sensitivities could yield highly efficient geometries in separating flows.

Appendices

Appendix A

Configuration and Spatial Verification of the Baselines

To ensure complete scientific reproducibility and to prevent the disruption of the narrative flow in Chapter 3, all comprehensive configuration matrices, domain truncation proofs, and spatial grid independence studies for the fluid benchmarks are documented in this appendix.

A.1 Verification Benchmark 1: Channel Flow

A.1.1 Physical and Geometric Parameters

The first verification benchmark consists of a pressure-driven periodic 2D channel. To align with fundamental turbulence research conventions, the characteristic length is defined as the full channel height (H), and the Reynolds number (Re_H) is computed strictly based on this geometric parameter rather than the hydraulic diameter. The complete physical setup is detailed in Table A.1.

A.1.2 Numerical Simulation Parameters

The channel flow was resolved using an Incremental Pressure Correction Scheme (IPCS) embedded within a Picard fixed-point iteration loop. The exact numerical solver configurations, convergence tolerances, and relaxation factors utilized in FEniCS are summarized in Table A.2.

A.1.3 Spatial Verification and Grid Independence

To guarantee that the numerical results were independent of the spatial discretization, a formal grid independence study was conducted. Three distinct boundary-layer inflated meshes (Coarse, Medium, and Fine) were evaluated. The geometric properties, node counts, and resolution parameters of this mesh hierarchy are explicitly detailed in Table A.3. Crucially, the first-layer height was kept constant across

TABLE A.1: Geometric, physical, and boundary-condition parameters for the periodic channel flow benchmark.

Parameter	Channel flow setting
Benchmark type	Pressure-driven, streamwise-periodic 2D channel
Domain dimensions	Length $L = 2.0$ m; full channel height $H = 2.0$ m
Characteristic length	Full channel height $H = 2.0$ m
Reference velocity	$U_{\text{ref}} = 18.5$ m/s, utilized to define the target Reynolds number
Kinematic viscosity	$\nu = 1.81818 \times 10^{-3}$ m ² /s
Reynolds number	$Re_H \approx 2.03 \times 10^4$
Body force	$\mathbf{f} = (0, 0)$ m/s ²
Driving condition	Fixed pressure drop from $p = 2$ Pa to $p = 0$ Pa across the periodic streamwise direction
Velocity boundary conditions	Periodic velocity between inlet/outlet planes; no-slip on top and bottom walls
Pressure boundary conditions	$p = 2$ Pa at the upstream pressure plane; $p = 0$ Pa at the downstream pressure plane
SA boundary conditions	Periodic $\tilde{\nu}$ between inlet/outlet planes; $\tilde{\nu} = 0$ at walls
Initial fields	$\mathbf{u}_0 = (18.5, 0)$ m/s, $p_0 = 2$ Pa, $\tilde{\nu}_0 = 5.0 \times 10^{-3}$ m ² /s
Verification mesh	Fine_WallResolved ; first layer $\Delta y_1 = 1.709 \times 10^{-3}$ m

all grids to strictly enforce the $y^+ \approx 1$ requirement, while the bulk and inflation-layer resolutions were systematically refined.

Visual confirmation of this spatial convergence is provided by the wall-normal velocity profiles plotted in Figure A.1. The **Fine_WallResolved** grid was consequently selected for the final verification against the Ansys Fluent baseline, while the coarser grids demonstrate that the wall-normal velocity profile is already asymptotically converged.

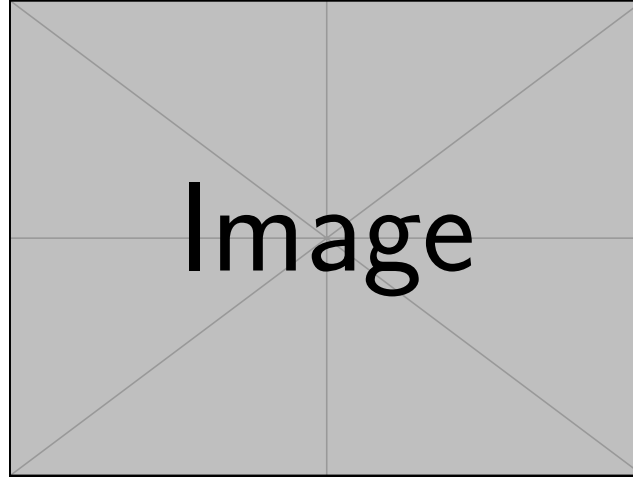


FIGURE A.1: Wall-normal velocity profiles for the Coarse, Medium, and Fine grids in the periodic channel flow benchmark. The overlapping profiles demonstrate asymptotic spatial convergence.

A. CONFIGURATION AND SPATIAL VERIFICATION OF THE BASELINES

TABLE A.2: Numerical simulation parameters for the FEniCS steady Spalart-Allmaras channel benchmark.

Parameter	Symbol	Setting
Velocity space	$\mathbf{u}$	Vector CG2, periodic in streamwise direction
Pressure space	p	Scalar CG1
SA working-variable space	$\tilde{\nu}$	Scalar CG1, periodic in streamwise direction
Quadrature degree	–	2
Maximum outer Picard steps	$N_{\text{Picard,max}}$	200
SA solves per Picard step	–	1
Outer velocity tolerance	$\epsilon_{\mathbf{u}}$	1.0×10^{-6}
Outer pressure tolerance	ϵ_p	1.0×10^{-6}
Outer SA tolerance	$\epsilon_{\tilde{\nu}}$	1.0×10^{-6}
IPCS pseudo-time step	Δt	5.0×10^{-3} s
Maximum IPCS steps per Picard step	–	500
IPCS velocity tolerance	–	1.0×10^{-6}
IPCS pressure tolerance	–	1.0×10^{-6}
Velocity relaxation factor	α_u	0.3
Pressure relaxation factor	α_p	0.3
SA relaxation factor	$\alpha_{\tilde{\nu}}$	0.3
SA lower bound	$\tilde{\nu}_{\min}$	1.0×10^{-12}
Wall-distance equation	–	Relaxed Eikonal, $\sigma_w = 0.1$, $G_0 = 20$
Wall-distance Newton settings	–	Relative tolerance 1.0×10^{-8} ; absolute tolerance 1.0×10^{-10} ; maximum 80 iterations; relaxation 0.5
Linear solver strategy	–	IPCS blocks: direct MUMPS; SA transport: default solver

TABLE A.3: Wall-resolved channel mesh hierarchy used for the grid-independence study. The first-layer height is kept fixed to maintain $y^+ \approx 1$, while the streamwise resolution, wall-normal resolution, and inflation-layer resolution are refined.

Parameter	Coarse	Medium	Fine
Mesh label	Coarse_WallResolved	Medium_WallResolved	Fine_WallResolved
Streamwise cells	10	20	40
Wall-normal cells	42	51	68
First-layer height Δy_1 [m]	1.709×10^{-3}	1.709×10^{-3}	1.709×10^{-3}
Minimum wall-normal spacing [m]	1.709×10^{-3}	1.709×10^{-3}	1.709×10^{-3}
Maximum wall-normal spacing [m]	7.707×10^{-2}	8.454×10^{-2}	7.589×10^{-2}
Inflation layers per wall	10	14	18
Growth ratio	1.45	1.35	1.25
Vertices	473	1,092	2,829
Triangular elements	840	2,040	5,440

A.2 Verification Benchmark 2: U-Bend Geometry

A.2.1 Physical and Geometric Parameters

The second verification benchmark evaluates a highly convective U-bend. Because this FEniCS domain is a 2D planar approximation of the symmetry mid-plane of the 3D Ansys VMFL048 case, the 3D physical pipe diameter (D) is explicitly retained as the characteristic length scale. This ensures the Reynolds (Re_D) and Dean (De) numbers remain non-dimensionally consistent with the commercial baseline. The configuration is detailed in Table A.4.

TABLE A.4: Geometric, physical, and boundary-condition parameters for the U-bend benchmark.

Parameter	U-bend setting
Benchmark type	Truncated 2D symmetry-plane U-bend based on Ansys VMFL048
Domain dimensions	Pipe diameter $D = 0.028$ m; centreline bend radius $R_c = 0.125$ m; straight-leg length $30D = 0.840$ m
Characteristic length	Physical pipe diameter $D = 0.028$ m
Reference / inlet velocity	Uniform prescribed inlet velocity $\mathbf{u}_{in} = (0, -1.42)$ m/s
Kinematic viscosity	$\nu = 8.9 \times 10^{-7}$ m ² /s
Reynolds number	$Re_D \approx 4.47 \times 10^4$
Dean number	$De \approx 1.50 \times 10^4$
Body force	$\mathbf{f} = (0, 0)$ m/s ²
Driving condition	Prescribed inlet velocity and zero relative static pressure at the outlet
Velocity boundary conditions	No-slip on all curved and straight pipe walls; natural velocity condition at pressure outlet
Pressure boundary conditions	$p = 0$ Pa at outlet; pressure left free at inlet and walls
SA boundary conditions	$\tilde{\nu}_{in} = 9.34 \times 10^{-5}$ m ² /s at inlet; $\tilde{\nu} = 0$ at walls; natural condition at outlet
Initial fields	$\mathbf{u}_0 = (0, 0)$ m/s, $p_0 = 0$ Pa, $\tilde{\nu}_0 = \tilde{\nu}_{in}$
Verification mesh	Medium_WallResolved; first layer $\Delta y_1 = 1.20 \times 10^{-5}$ m

A.2.2 Numerical Simulation Parameters

Due to the adverse pressure gradients and secondary flow separation induced by the streamline curvature, the U-bend represents a significantly more demanding numerical challenge than the channel. Consequently, the FEniCS solver required a smaller pseudo-time step, tighter under-relaxation, and advanced iterative preconditioners to maintain stability, as detailed in Table A.5.

A.2.3 Domain Truncation Study

The original 3D Ansys VMFL048 benchmark utilizes highly extended straight legs measuring 1.555 m to ensure fully developed flow. To reduce computational over-

A. CONFIGURATION AND SPATIAL VERIFICATION OF THE BASELINES

TABLE A.5: Numerical simulation parameters for the highly convective FEniCS U-bend benchmark.

Parameter	Symbol	Setting
Velocity space	$\mathbf{u}$	Vector CG2
Pressure space	p	Scalar CG1
SA working-variable space	$\tilde{v}$	Scalar CG1
Quadrature degree	–	4
Maximum outer Picard steps	$N_{\text{Picard,max}}$	180
SA solves per Picard step	–	1
Outer velocity tolerance	$\epsilon_{\mathbf{u}}$	1.0×10^{-4}
Outer pressure tolerance	ϵ_p	1.0×10^{-4}
Outer SA tolerance	$\epsilon_{\tilde{v}}$	1.0×10^{-6}
IPCS pseudo-time step	Δt	1.0×10^{-4} s
Maximum IPCS steps per Picard step	–	180
IPCS velocity tolerance	–	1.0×10^{-4}
IPCS pressure tolerance	–	2.0×10^{-3}
Velocity relaxation factor	α_u	0.3
Pressure relaxation factor	α_p	0.1
SA relaxation factor	$\alpha_{\tilde{v}}$	0.01
SA lower bound	$\tilde{v}_{\min}$	1.0×10^{-12}
Wall-distance equation	–	Relaxed Eikonal, $\sigma_w = 0.1$, $G_0 = 20$
Wall-distance Newton settings	–	Relative tolerance 1.0×10^{-8} ; absolute tolerance 1.0×10^{-10} ; maximum 80 iterations; relaxation 0.5
Linear solver strategy	–	IPCS blocks: GMRES with ILU velocity/correction preconditioning and Hypre-AMG pressure preconditioning; SA transport: default solver

head for the topology optimization pipeline, these legs were truncated to $30D$ (0.840 m). A preliminary sensitivity study was conducted in Ansys Fluent, comparing the full-length and truncated geometries. To mathematically prove that this reduction does not artificially alter the flow physics, the cross-sectional velocity profiles at the critical bend exit were extracted for both domains, as plotted in Figure A.2. The complete overlap of the profiles confirms that the $30D$ truncation successfully isolated the curvature physics without artificially impacting the boundary layer separation.

A.2.4 Spatial Verification and Grid Independence

Managing the highly stretched, non-orthogonal inflation layers around the curved arcs of the U-bend required custom mesh generation via Gmsh. A spatial grid independence study was conducted across three refined unstructured grids. The node counts, element densities, and inflation layer parameters for these generated meshes are explicitly documented in Table A.6. Similar to the channel benchmark, the wall-

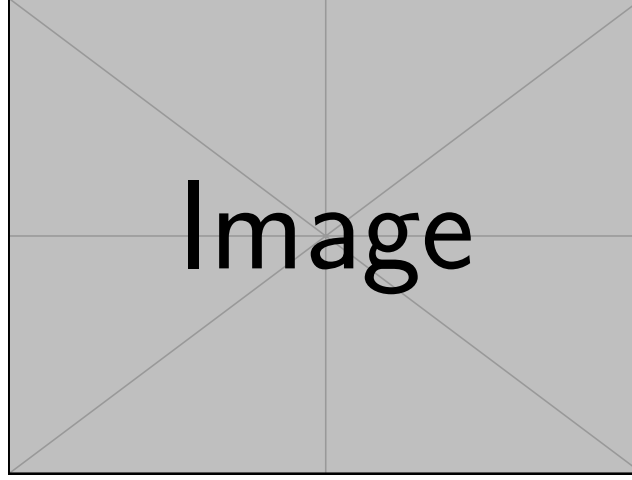


FIGURE A.2: Velocity profile comparison at the U-bend exit between the full Ansys domain (1.555 m legs) and the truncated topology optimization domain (30D legs). The complete overlap confirms the validity of the truncated geometry.

adjacent element height was strictly maintained at 1.20×10^{-5} m to satisfy the low-Reynolds SA wall constraints, while the internal element sizing was systematically refined.

TABLE A.6: Wall-resolved U-bend mesh hierarchy used for the grid-independence study. The first-layer height is kept fixed to maintain $y^+ \approx 1$, while the bulk element size and inflation-layer resolution are refined.

Parameter	Coarse	Medium	Fine
Mesh label	Coarse_WallResolved	Medium_WallResolved	Fine_WallResolved
Bulk element size [m]	2.80×10^{-3}	1.40×10^{-3}	8.00×10^{-4}
First-layer height Δy_1 [m]	1.20×10^{-5}	1.20×10^{-5}	1.20×10^{-5}
Inflation layers	10	14	18
Growth ratio	1.45	1.30	1.25
Inflation thickness [m]	1.07×10^{-3}	1.53×10^{-3}	2.62×10^{-3}
Vertices	23,751	73,978	182,450
Triangular elements	45,998	144,952	359,646

Convergence was visually verified by extracting the velocity profiles directly at the bend exit, a highly sensitive region for curvature-induced separation, as illustrated in Figure A.3. The chosen Medium_WallResolved grid perfectly captured the velocity gradients near the wall and provided sufficient isotropic resolution in the bulk flow to resolve the separation bubbles without numerical instability.

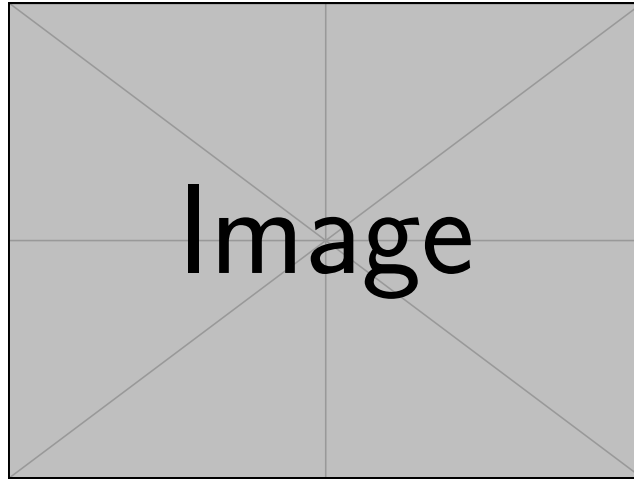


FIGURE A.3: Velocity profiles at the U-bend exit for the Coarse, Medium, and Fine grids. The profiles demonstrate spatial convergence, justifying the selection of the Medium grid for the final verification.

Appendix B

Mesh Independence of the Extracted Validation Geometries

To ensure complete scientific reproducibility and support the cross-code validation campaign detailed in Chapter 5, this appendix documents the spatial grid independence studies for all extracted Ansys geometries. Because the native FEniCS topology optimization utilizes a fixed, Brinkman-penalized Cartesian grid, the transition to a sharp, body-fitted CAD geometry introduces potential spatial discretization errors. Establishing grid independence ensures that the reported pressure drops (Δp) accurately reflect the underlying solver physics and turbulence models, rather than numerical remeshing noise.

B.1 Near-Wall Treatment and Meshing Strategy

The extracted validation geometries must satisfy the strict near-wall discretization requirements of the selected commercial turbulence closures. Both the Spalart-Allmaras (SA) and SST $k-\omega$ models utilized in the Ansys Fluent re-analysis integrate the governing transport equations completely through the viscous sublayer to the solid boundary. This approach strictly circumvents the use of empirical wall functions, necessitating a highly controlled discretization of the boundary layer.

For all evaluated geometries, a hybrid meshing strategy was deployed. The bulk flow domains were discretized utilizing unstructured polyhedral elements to efficiently manage complex, optimized internal curves. At all extracted solid boundaries ($\bar{\gamma} = 0$), explicit inflation layers were generated. To guarantee accurate resolution of the severe velocity gradients within the viscous sublayer, the first-layer cell height (Δy_1) was calibrated to maintain a non-dimensional wall distance of $y^+ \leq 1$ across all solid surfaces. Subsequent prism layers were generated with a controlled geometric growth rate not exceeding 1.2 to ensure a smooth volumetric transition into the bulk mesh.

B.2 Grid Convergence Analysis

To quantify spatial discretization error, three successive meshes (Coarse, Medium, and Fine) were generated for each benchmark case. The global spatial refinement factor between each successive mesh level was maintained at approximately $r \approx 1.5$. The total static pressure drop (Δp) across the design domain was selected as the macroscopic convergence metric. The resulting mesh statistics and convergence behavior are documented in Table B.1.

TABLE B.1: Grid convergence study for the extracted validation geometries. Metrics are reported for both the Ansys SA and SST $k-\omega$ models under identical boundary conditions.

Case	Model	Coarse Elements	Medium Elements	Fine Elements	Relative Δp Change (Med-Fine)
Diffuser	SA	[TBD]	[TBD]	[TBD]	[TBD]%
Diffuser	SST $k-\omega$	[TBD]	[TBD]	[TBD]	[TBD]%
Double pipe	SA	[TBD]	[TBD]	[TBD]	[TBD]%
Double pipe	SST $k-\omega$	[TBD]	[TBD]	[TBD]	[TBD]%
Curved benchmark	SA	[TBD]	[TBD]	[TBD]	[TBD]%
Curved benchmark	SST $k-\omega$	[TBD]	[TBD]	[TBD]	[TBD]%

As indicated by the convergence data in Table B.1 and the plotted trends in Figure B.1, the macroscopic pressure drop exhibits clear asymptotic behavior as the spatial resolution increases. Across all benchmarks, the relative discrepancy between the Medium and Fine meshes falls below the strict numerical threshold of [Insert %, e.g., 1.5%], indicating that the governing equations are sufficiently resolved. Consequently, the spatial resolution corresponding to the **Medium** mesh profile was selected as the standard for all comparative CFD validation campaigns in this thesis.

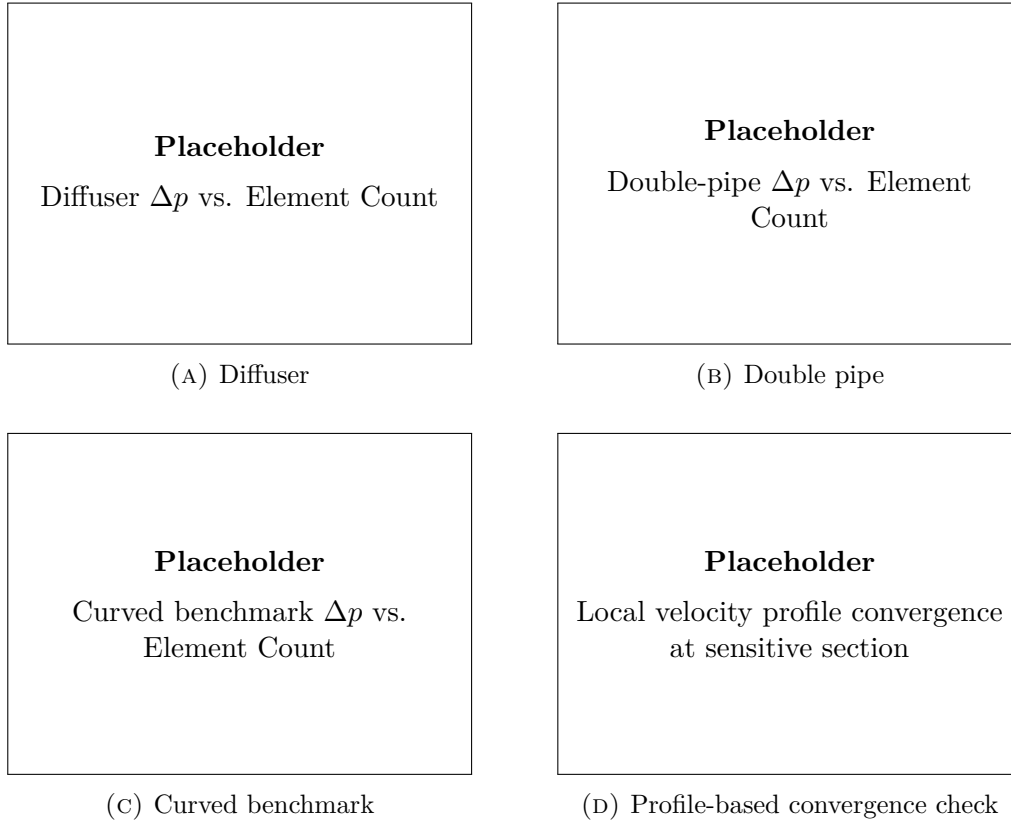


FIGURE B.1: Asymptotic convergence of the macroscopic pressure drop metric and localized velocity profiles across successive mesh refinement levels.

Appendix C

Comparison of Optimized Designs with Laminar Baselines

While Chapter 5 successfully verifies the computational accuracy of the fully coupled FEniCS-SA framework against commercial CFD solvers, an essential engineering justification underpins this methodology: does optimizing a fluid manifold under turbulent conditions yield a physically superior design compared to a standard laminar baseline? Because continuous turbulent adjoint pipelines demand substantially more computational resources, tighter iterative coupling, and rigorous stabilization compared to their laminar equivalents, this mathematical complexity must translate into a measurable reduction in hydrodynamic losses. This appendix quantifies that performance gap.

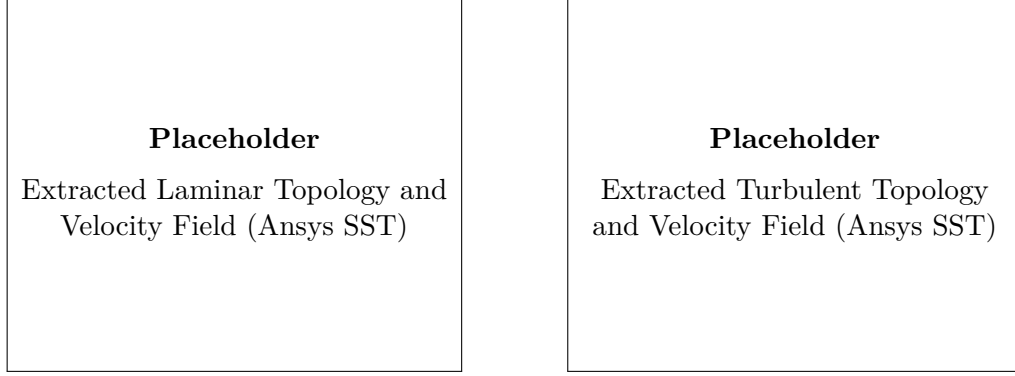
C.1 Methodology and Evaluation Pipeline

To conduct a rigorous comparative analysis, a representative routing domain (the [Insert Benchmark] case) was optimized twice utilizing the custom FEniCS framework. The first design iteration was generated using the steady-state, incompressible laminar Navier-Stokes equations, effectively blinding the optimizer to turbulent diffusion and Reynolds stresses. The second iteration utilized the fully coupled Spalart-Allmaras methodology developed in Chapter 4.

Both resulting continuous design fields were subjected to the identical geometry extraction and thresholding procedure. To evaluate their true operational performance, the sharp CAD manifolds were imported into Ansys Fluent, meshed with wall-resolved inflation layers ($y^+ \leq 1$), and subjected to identical turbulent boundary conditions utilizing the SST k - ω turbulence model. The inlets were prescribed a uniform velocity profile of [Insert Velocity] m/s and a turbulence intensity of [Insert TI]%, corresponding to a highly convective operational Reynolds number of $Re = [\text{Insert Re}]$.

C.2 Morphological and Hydrodynamic Comparison

The geometric discrepancies between the laminar and turbulent topologies are stark, directly reflecting the distinct physical phenomena captured by the respective adjoint sensitivities. Figure C.1 illustrates the velocity magnitude contours for both geometries evaluated under the high-Reynolds boundary conditions.



(A) Laminar-optimized design evaluated under turbulent flow.

(B) Turbulent-optimized design (SA) evaluated under turbulent flow.

FIGURE C.1: Velocity magnitude contours of the laminar and turbulent designs, evaluated under identical high-Reynolds boundary conditions in Ansys Fluent (SST $k-\omega$).

As observed in Figure C.1(a), the laminar optimizer favors abrupt expansions and sharp internal routing. Because the laminar adjoint formulation lacks the convective momentum transport and severe boundary layer mechanics associated with inertial flows, it distributes structural material strictly to minimize bulk viscous friction. Consequently, when this laminar geometry operates in a turbulent regime, the severe adverse pressure gradients trigger massive boundary layer separation, generating highly dissipative recirculation zones.

Conversely, Figure C.1(b) demonstrates the mathematical response of the turbulent optimizer. The SA-driven framework actively suppresses separation by generating heavily streamlined, gradual geometric transitions. It trades a minor increase in wetted surface area (and subsequent skin friction) for a drastic reduction in form drag and turbulent wake generation.

C.3 Quantitative Performance Analysis

The physical differences in flow morphology translate directly into macroscopic thermodynamic performance. Table C.1 summarizes the power dissipation and peak turbulence metrics evaluated by the Ansys SST $k-\omega$ solver.

The turbulent-optimized manifold exhibits a significant [Insert %] reduction in total pressure drop compared to the laminar baseline. This severe performance penalty in the laminar geometry is heavily dominated by the artificial turbulent ki-

C. COMPARISON OF OPTIMIZED DESIGNS WITH LAMINAR BASELINES

TABLE C.1: Quantitative performance metrics of the laminar and turbulent optimized topologies evaluated under identical turbulent boundary conditions (Ansys SST $k-\omega$).

Design Origin	Total Pressure Drop (Δp) [Pa]	Peak Eddy Viscosity Ratio	Relative Performance
Laminar TO Formulation	[TBD]	[TBD]	Baseline
Turbulent TO Formulation (SA)	[TBD]	[TBD]	- [TBD]%

netic energy generated within its flow separation zones. Ultimately, this independent CFD re-analysis confirms that extending topology optimization into the turbulent regime is not merely an academic exercise, but a strict engineering requirement to ensure the manufacturability and efficiency of high-Reynolds-number fluid systems.

Bibliography

- [1] J. Alexandersen. A detailed introduction to density-based topology optimisation of fluid flow problems with implementation in matlab. *Structural and Multidisciplinary Optimization*, 66(1):12, 2023.
- [2] J. Alexandersen and C. S. Andreasen. A review of topology optimisation for fluid-based problems. *Fluids*, 5(1):29, 2020.
- [3] M. S. Alnæs, J. Blechta, J. Hake, A. Johansson, B. Kehlet, A. Logg, C. Richardson, J. Ring, M. E. Rognes, and G. N. Wells. The fenics project version 1.5. *Archive of Numerical Software*, 3(100), 2015.
- [4] D. H. Alonso, L. F. Garcia Rodriguez, and E. C. N. Silva. Flexible framework for fluid topology optimization with openfoam and finite element-based high-level discrete adjoint method (fenics/dolfin-adjoint). *Structural and Multidisciplinary Optimization*, 64(6):4409–4440, 2021.
- [5] Altair Engineering, Inc. *AcuSolve Turbulence Modeling Training Manual*, 2023. Available online: <https://help.altair.com/hwcfdsolvers/acusolve>.
- [6] E. Andreassen, A. Clausen, M. Schevenels, B. S. Lazarov, and O. Sigmund. Efficient topology optimization in matlab using 88 lines of code. *Structural and Multidisciplinary Optimization*, 43(1):1–16, 2011.
- [7] ANSYS, Inc., Canonsburg, PA. *Ansys Fluid Dynamics Verification Manual, Release 2024 R1*, January 2024.
- [8] R. K. Apalowo. Numerical study of different models for turbulent flow in 90° pipe bend. *U.Porto Journal of Engineering*, 8(2):121–134, 2022.
- [9] D. N. Arnold, F. Brezzi, B. Cockburn, and L. D. Marini. Unified analysis of discontinuous galerkin methods for elliptic problems. *SIAM Journal on Numerical Analysis*, 39(5):1749–1779, 2002.
- [10] A. Bayat, H. Li, and J. Alexandersen. Density-based topology optimization for turbulent fluid flow using the standard k-epsilon rans model with wall-functions imposed through an implicit wall penalty formulation. arXiv preprint arXiv:2601.02202, Jan. 2026. Published 5 January 2026.

- [11] M. P. Bendsøe and N. Kikuchi. Generating optimal topologies in structural design using a homogenization method. *Computer methods in applied mechanics and engineering*, 71(2):197–224, 1988.
- [12] T. Borrvall and J. Petersson. Topology optimization of fluids in stokes flow. *International journal for numerical methods in fluids*, 41(1):77–107, 2003.
- [13] A. J. Chorin. Numerical solution of the navier-stokes equations. *Mathematics of computation*, 22(104):745–762, 1968.
- [14] Y. Deng, Z. Liu, and Y. Wu. Topology optimization of steady and unsteady incompressible navier-stokes flows driven by body forces. *Structural and Multidisciplinary Optimization*, 47:555–570, 2013.
- [15] C. B. Dilgen, S. B. Dilgen, D. R. Fuhrman, O. Sigmund, and B. S. Lazarov. Topology optimization of turbulent flows. *Computer Methods in Applied Mechanics and Engineering*, 331:363–393, 2018.
- [16] A. Fawaz, Y. Hua, S. Le Corre, Y. Fan, and L. Luo. Topology optimization of heat exchangers: A review. *Energy*, 258:124053, 2022.
- [17] J. H. Ferziger, M. Perić, and R. L. Street. *Computational Methods for Fluid Dynamics*. Springer, Berlin, Heidelberg, 4th edition, 2012.
- [18] A. Gersborg-Hansen, M. P. Bendsøe, and O. Sigmund. Topology optimization of channel flow problems. *Structural and multidisciplinary optimization*, 30(3):181–192, 2005.
- [19] C. Geuzaine and J.-F. Remacle. Gmsh: A 3-d finite element mesh generator with built-in pre-and post-processing facilities. *International journal for numerical methods in engineering*, 79(11):1309–1331, 2009.
- [20] J. S. Hesthaven and T. Warburton. *Nodal Discontinuous Galerkin Methods: Algorithms, Analysis, and Applications*. Springer Science & Business Media, New York, NY, 2007.
- [21] joove123. k-epsilon fenics implementation. <https://github.com/joove123/k-epsilon.git>, 2026.
- [22] H. Kobayashi, K. Yaji, S. Yamasaki, and K. Fujita. Topology design of two-fluid heat exchange. *Structural and Multidisciplinary Optimization*, 63:821–834, 2021.
- [23] H. P. Langtangen and A. Logg. *Solving PDEs in Python: The FEniCS Tutorial I*. Springer Nature, 2016.
- [24] B. S. Lazarov and O. Sigmund. Filters in topology optimization based on helmholtz-type differential equations. *International Journal for Numerical Methods in Engineering*, 86(6):765–781, 2011.

- [25] A. Logg, K.-A. Mardal, and G. N. Wells. *Automated Solution of Differential Equations by the Finite Element Method: The FEniCS Book*, volume 84. Springer Science & Business Media, 2012.
- [26] J. Marcibál. Turbulence modeling in computational fluid dynamics. Master’s thesis, University of Southern Denmark, 6 2024.
- [27] A.-S. Margetis, E. Papoutsis-Kiachagias, and K. Giannakoglou. The continuous adjoint to the incompressible (d) des spalart-allmaras turbulence models. *Computers & Fluids*, 284:106439, 2024.
- [28] C. Othmer. Adjoint methods for car aerodynamics. *Journal of Mathematics in Industry*, 4(1):1–25, 2014.
- [29] E. M. Papoutsis-Kiachagias and K. C. Giannakoglou. Continuous adjoint methods for turbulent flows, applied to shape and topology optimization: industrial applications. *Archives of Computational Methods in Engineering*, 23(2):255–299, 2016.
- [30] S. B. Pope. *Turbulent Flows*. Cambridge University Press, 2000.
- [31] C. L. Rumsey et al. Turbulence modeling resource. NASA Langley Research Center, <https://turbmodels.larc.nasa.gov>, 2023. Accessed: 2024-05-15.
- [32] C. L. Rumsey, T. B. Gatski, and J. H. Morrison. Turbulence model predictions of strongly curved flow in a U-Duct. *AIAA Journal*, 38(8):1394–1402, 2000.
- [33] M. Schramm, B. Stoevesandt, and J. Peinke. Optimization of airfoils using the adjoint approach and the influence of adjoint turbulent viscosity. *Computation*, 6(1):10, 2018.
- [34] P. Spalart and S. Allmaras. A one-equation turbulence model for aerodynamic flows. *30th aerospace sciences meeting and exhibit*, page 439, 1992.
- [35] P. R. Spalart and S. R. Allmaras. A one-equation turbulence model for aerodynamic flows. *La Recherche Aéronautique*, 1:5–21, 1994.
- [36] K. Svanberg. The method of moving asymptotes—a new method for structural optimization. *International journal for numerical methods in engineering*, 24(2):359–373, 1987.
- [37] R. Temam. Sur l’approximation de la solution des équations de navier-stokes par la méthode des pas fractionnaires (i). *Archive for Rational Mechanics and Analysis*, 32(2):135–153, 1969.
- [38] P. Tucker. Differential equation-based wall distance computation for des and rans. *Journal of Computational Physics*, 190(1):229–248, 2003.
- [39] H. K. Versteeg and W. Malalasekera. *An introduction to computational fluid dynamics: the finite volume method*. Pearson education, 2007.

- [40] F. M. White and J. Majdalani. *Viscous fluid flow*, volume 3. McGraw-Hill New York, 2006.
- [41] H. Wu, D. Wang, and X. Huang. Effect of constant eddy viscosity assumption on optimization using a discrete adjoint method. *Chinese Journal of Aeronautics*, 36(11):102–118, 2023.
- [42] G. H. Yoon. Topology optimization for turbulent flow with spalart-allmaras model. *Computer Methods in Applied Mechanics and Engineering*, 303:288–311, 2016.
- [43] J. Zhao, M. Zhang, Y. Zhu, R. Cheng, and L. Wang. Topology optimization of turbulent forced convective heat sinks using a multi-layer thermofluid model. *Structural and Multidisciplinary Optimization*, 64(6):3835–3859, 2021.
- [44] T. Zhou, T. Liu, Y. Deng, L. Chen, S. Qian, and Z. Liu. Design of microfluidic channel networks with specified output flow rates using the cfd-based optimization method. *Microfluidics and Nanofluidics*, 21(1), 2017.
- [45] A. Zymaris, D. Papadimitriou, K. Giannakoglou, and C. Othmer. Continuous adjoint approach to the spalart–allmaras turbulence model for incompressible flows. *Computers & Fluids*, 38(8):1528–1538, 2009.

Code of conduct and transparency statement on the use of GenAI for KU Leuven-students (academic year 2025–2026)

Generative AI (GenAI) assistance tools can be used to generate various types of content, including text, images, code, video, music, or combinations thereof. Common examples of such tools include ChatGPT, Google Gemini, Microsoft Copilot, Midjourney, Claude.ai, Perplexity.ai, and DALL·E, among others.

This code of conduct is a tool that helps students to be transparent about the use of GenAI and fits within the university's principles on academic integrity.

Important guidelines and remarks

- **Sensitive or personal data:** Some GenAI tools protect your input and sensitive or personal data better than others. There is often no transparency on what the owners of the AI applications do with the data entered. Therefore, **do not enter sensitive or personal data in free GenAI tools**. More info about what sensitive and personal data are, is described in the [three confidentiality levels of the KU Leuven data classification model](#) (non-confidential, confidential, strictly confidential). For confidential data you should preferably use **M365 Copilot Chat with your KU Leuven account**. If you use another GenAI tool, you must be **absolutely certain** it does **not store or reuse the data you enter**. **Try to avoid entering these data. Do so only if strictly necessary, and only to the extent required. For personal data, work with anonymous or pseudonymized data.** Additionally, in case of strictly confidential data this should first be discussed with the teaching staff of the course or your thesis supervisor.
- **Copyrighted materials:** For lawfully obtained copyrighted material you should preferably use **M365 Copilot Chat with your KU Leuven**

account. If you use another GenAI tool, you must be **absolutely certain** it does **not store or reuse the data you enter**.

- GenAI assistance may not be used for data or topics covered by a **Non-Disclosure Agreement (NDA)**. Even in the absence of an NDA, certain information may still need to be treated as confidential—for example, due to regulatory requirements or the risk of significant harm to the university if disclosed. In case of doubt, check with your teaching staff or supervisor.
- If your master’s thesis is under **embargo**, you should first discuss with your supervisor whether the use of GenAI is permitted.
- Before using a GenAI tool, always consider whether its use is responsible, including from a **sustainability perspective** (e.g. when using GenAI as a search engine, language assistant, ...).
- **Take a scientific and critical attitude** when interacting with GenAI assistance and interpreting its output, that may not always be correct.
- As a student you are responsible for complying with Article 84 of the Regulations on Education and Examinations: your report or thesis should reflect your own knowledge, understanding and skills. Be aware that plagiarism rules also apply to (work that is the result of) the use of GenAI assistance tools.

Exam Regulations Article 84: *“Every conduct individual students display with which they (partially) inhibit or attempt to inhibit a correct judgement of their own knowledge, understanding and/or skills or those of other students, is considered an irregularity which may result in a suitable penalty. A special type of irregularity is plagiarism, i.e. copying the work (ideas, texts, structures, designs, images, plans, codes , ...) of others or prior personal work in an exact or slightly modified way without adequately acknowledging the sources. Every possession of prohibited resources during an examination (see article 65) is considered an irregularity.”*

- In order to maintain academic integrity and avoid plagiarism, **more information about being transparent on the use of GenAI assistance and about correctly citing and referencing GenAI** can be found on this website for students (**Dutch/English**).
- **Additional reading: KU Leuven guidelines on responsible use of Generative AI tools, and other information** (**Dutch/English**)

A few final words

If you are uncertain whether or not you should declare your use of GenAI tools, we suggest that you discuss this with your instructor or supervisor. It is always safer to declare GenAI use, even when it is not strictly required. However, declaring GenAI use does not entail that its use is

allowed; the right column in the table below provides more detailed instructions in this regard (code of conduct).

Moreover, advanced AI tools are evolving rapidly, and their capabilities have expanded significantly in a short period of time. As a result, we do not yet have all the answers about their responsible use. Finally, it is important to follow-up on the most recent evolutions in AI technologies, to have an open mind but also to be a bit cautious, to communicate with instructors, teaching assistants, supervisors and peers, to be as transparent as we can, and to learn together as we move along.

Student name:

Student number:

Please indicate with "X" whether it relates to a course assignment or to the Bachelor's or Master's thesis:

☐ This form is related to a **course assignment**.

Course name: **Course number:**

This form is related to my ☐ **Bachelor's** or ☐ **Master's thesis**.

Title Bachelor's or Master's thesis: **Supervisor:**

Daily supervisor:

Please indicate with "X":

☐ I **did not use** any GenAI assistance tool.

☐ I **did use** GenAI Assistance. In this case **specify which ones** (e.g. ChatGPT, M365 Copilot, ...):

GenAI assistance used as/for:	Name of the GenAI tool(s) used If helpful, also describe in which way you were using GenAI related to what is specified as code of conduct.	Code of conduct: For each of the categories below, always take into account the important guidelines and remarks mentioned above (e.g. copyrighted data, sensitive or personal data,...).
As a language assistant for reviewing or improving texts I wrote myself	<i>add text here</i>	This use is similar to using spelling and grammar check tools. In general, you do not have to refer to such kind of GenAI use in the text. However, be careful: When using GenAI tools on texts you did not write yourself to improve the text, you have to refer to the original source or author, otherwise you are committing plagiarism and thus an irregularity.
As a paraphrasing tool	<i>add text here</i>	This use is allowed except when it is prohibited by the teacher or the program of study. You may paraphrase your own text or texts by an author other than yourself and take inspiration from what a GenAI tool or another tool suggests (unless it is not allowed). In general, you do not have to refer to such kind of GenAI use or other paraphrasing tools in the text. However, be careful: If it entails text by an author other than yourself, you are not allowed to include that paraphrased text without reference to the original source or author. Without such reference, you would be committing plagiarism and thus an irregularity.

For translation aid to improve texts I wrote myself or to better understand text from others	<i>add text here</i>	This use is allowed except when it is prohibited by the teacher or the program of study. It is similar to using translation tools (Google translate, DeepL, ...). In general, you do not have to refer to such kind of GenAI use in the text. However, be careful: • You are not allowed to include that translated text without reference to the original source or author. Without such reference, you would be committing plagiarism and thus an irregularity. • Always check the translated text for correctness and meaning.
As a search engine to get information on a topic or to search for existing research on the topic	<i>add text here</i>	This use is similar to e.g. a Google search or checking Wikipedia. If you write your own text based on this information, you do not have to refer to the use of GenAI in the text. You only have to refer to the existing research and references you have checked and used (without such references you would be committing plagiarism and thus committing an irregularity). However, be careful: • Be aware that the output of the GenAI tool cannot be guaranteed as a 100% reliable source of information. The output may not be entirely correct and/or be limited due to the databases it uses. Moreover, knowledge evolves and may change over time; therefore, the database of the GenAI tool may not be up to date. Therefore, verify the information and do not just copy-paste it as you should understand and critically process everything you are writing.

For literature search	<i>add text here</i>	This use is comparable to e.g. a Google Scholar search. You do not have to refer to such kind of GenAI use; you only have to refer to the literature references you have checked and used (without such references you would be committing plagiarism and thus committing an irregularity). However, be careful: • Be aware that the search output is restricted to the database the GenAI tool is built on. After this initial search, look for scientific sources and conduct your own analysis of the source documents. Interpret, analyse and process the information you obtained; verify it and do not just copy-paste it as you should understand and critically process everything you are writing. • Be aware that some GenAI tools may output no or wrong references. As a student you are responsible for further checking and verifying the absence or correctness of references; do not just copy-paste it.
For generating programming code	<i>add text here</i>	Use of GenAI for coding is allowed except when it is prohibited by the teacher or the program of study. If used for coding, correctly mention the use of GenAI assistance in accordance with the instructions on the page on being transparent about the use of GenAI.

For new ideas generating (research)	<i>add text here</i>	This use of GenAI is allowed except when it is prohibited by the teacher or the program of study. Correctly mention the use of GenAI assistance in accordance with the instructions on the page on being transparent about the use of GenAI. Be careful: • Further verify in this case whether the idea is novel or not. It is likely that it is related to existing work. If so, that existing work should be correctly referenced in the text (without such reference you would be committing plagiarism and thus committing an irregularity).
For generating synthetic data	<i>add text here</i>	Use of GenAI for generating synthetic data is allowed, provided that it is methodologically and ethically justifiable, except when it is prohibited by the teacher or the program of study. Always correctly mention the use of GenAI assistance in accordance with the instructions on the page on being transparent about the use of GenAI. Be careful: • Always carefully evaluate the generated synthetic data for quality and possible bias since the output is highly dependent on the quality of the data on which the models are trained.

For generating blocks of text (other than the allowed use without referencing mentioned above)		According to Article 84 of the Regulations on Education and Examinations your text should allow to correctly and properly assess your own knowledge, understanding and skills. Therefore, inserting blocks of text without quotes and a reference to GenAI assistance in your work is not allowed. Be careful: • If it is really needed to insert a block of text from a GenAI tool, for instance because of the nature of your assignment, mention it as a citation by using quotes and correctly mention the use of GenAI assistance in accordance with the instructions page on being transparent about the use of GenAI. • However, in general, such GenAI use should be kept to an absolute minimum; you should always check the original sources.
For generating visuals, video or audio	<i>add text here</i>	Use of GenAI for generating visuals, audio or video is allowed except when it is prohibited by the teacher or the program of study. Be careful: • If used, refer to GenAI for visuals according to the usual referencing style, following the instructions on the page on referencing GenAI. • If you work with existing visuals, audio or video, that existing work should be correctly referenced in the text (without such reference you would be committing plagiarism and thus an irregularity). • Explain the usage in the methods section (if there is one) and optionally attach (or link to) the prompts with the full output (or history).

Other use (specify here; this may also include a combination of types of use mentioned above):	<i>add text here</i>	To make sure other use of GenAI is allowed within the course or thesis, and if so, the conditions that may apply, contact the teaching staff of the course or the supervisor of the thesis beforehand and explain the intended GenAI use. Also inform the programme director. Motivate how you would comply with Article 84 of the exam regulations. Explain the use and the added value of the AI tool you consider to use and how it is in accordance with the assignment or thesis and the KU Leuven guidelines on responsible use of Generative AI tools. Depending on the kind of GenAI use, it may be needed to properly reference it in the text, in accordance with the instructions page on being transparent about the use of GenAI.
--	-----------------------------	--